Cross-chain Lightning Trades: Getting the Advantages of a Custodial Exchange while Keeping Your Assets

Michele Ciampi[1], Muhammad Ishaq[2], Rafail Ostrovsky[3], Ioannis Tzannetos[4,5], and Vassilis Zikas[6]

[1]University of Edinburgh
[2]Sunday Group Inc
[3]University of California, Los Angeles
[4]Purdue University
[5]National Technical University of Athens
[6]Georgia Institute of Technology
michele.ciampi@ed.ac.uk, ishaq@sundaygroupinc.com, rafail@cs.ucla.edu, itzannet@purdue.edu, vzikas@gatech.edu

Abstract

Payment channels are a cornerstone of a scalable blockchain infrastructure that enables transacting parties to lock assets on the blockchain and perform rapid off-chain updates with minimal latency and overhead. These protocols dramatically reduce on-chain interaction and improve throughput, with blockchain consensus only invoked in the event of disputes or final closure. While widely adopted in single-chain settings—such as in the Lightning Network for Bitcoin—existing constructions have several limitations, in particular they suffer from at least one of the following limitations:

1. *No cross-chain.* They do not enable fast trading of assets that reside on multiple isolated blockchains.
2. *Non-optimal round complexity.* The off-chain round complexity is not optimal (i.e., parties require more than two rounds to update the channel).
3. *No bitcoin compatibility.* They require a more advanced scripting language that prevents cross-chain interaction between chains that only have a simple (Bitcoin-like) scripting language.

In this work, we introduce a novel payment channel protocol that breaks through all the above limitations. Our construction supports bidirectional, multiasset, and off-chain interaction across an arbitrary set of blockchains, where each update of the channel requires the parties to send only one message each. Our protocol is fully compatible with Bitcoin and can be deployed on chains with only minimal scripting capabilities, making it broadly applicable to real-world blockchain networks.

Crucially, our design ensures atomic settlement across blockchains *without* relying on trusted intermediaries. We formally prove the security of our protocol together with a novel definition of the cross-chain payment channel that we introduce. Finally, we empirically validate our protocol, showing the minimal costs that our payment channel incurs in the setting where two parties make multiple exchanges of assets that reside on three different blockchains.

1 Introduction

Decentralized Finance (DeFi) is one of the core applications of decentralized blockchain systems. Its promise is to disrupt the traditional financial sector by offering similar services on digital assets. Among the plethora of DeFi applications proposed, two stand out as particularly relevant to our work: *Atomic Swaps* and *Payment Channel (PC).*

Atomic Swaps (AS) [Han+24; Her18; TMM22; BDM16; Cam+17; Fuc19; EFS20; DEF18; DH20] allow two parties, typically referred to as Alice and Bob, to exchange digital assets that reside on (different) blockchains, in such a way that either the swap is executed and both parties receive the agreed-upon outcome, or the exchange does not occur at all. This all-or-nothing property is enforced through cryptographic mechanisms, which ensure that both participants must fulfill certain conditions within a predefined time window in order for the swap to succeed. If these conditions are not met, the assets are refunded to their original owners.

Payment Channel (PC) [Tod14; PD16; DW15; DRO18; TMSS23; Aum+21b; Kur+; Aum+22; Thy+20; Mor+20; Lin+; Cha+21b; Ava+21; Mir+22a; Aum+24; Qin+23] is an offline payment mechanism that works as follows: The two parties, Alice and Bob, open the PC by posting, on chain, a *funding* transaction that transfers an appropriate amount of their funds to an address that they control together. The idea is that neither of them can unilaterally withdraw funds from this shared address. Once the funding transaction has settled on-chain, the channel is considered *open*. Alice and Bob can now collaborate off-chain to *update* the distribution of funds in the channel by signing appropriate payment transactions together. The money flow in an update is restricted to channel's directionality. In a *uni-directional* channel, every update decreases Alice's (sender's) balance and increases Bob's (receiver's) balance i.e., money flows from Alice to Bob. On the other hand, in a *bi-directional* channel, money can flow in either direction. Alice and Bob may update the channel several times off of the chain. When it is time to *close* the channel, one of them posts the latest update to the blockchain. *Punishment* mechanisms ensure that dishonest behavior (e.g., posting a stale/older update) is disincentivized. The advantage of payment channels is that once a channel is open, the involved parties may transact a number of times between them without the need to post corresponding transactions on the blockchain. They are (1) not limited by the blockchain throughput or speed, and (2) save on the blockchain fees by moving most of the work off-chain.

In this work, we want to get all the benefits (and more) of both the concept of payment channels and cross-chain atomic swaps by devising a novel payment channel protocol where parties can exchange assets residing on multiple blockchains in a fair manner (like in the case of atomic swaps) with a throughput comparable to existing payment channel solutions.

In particular, we consider the two-party setting, where one party, Alice (or *Proposer*), can transfer the entire information needed to update the channel state in a single round. The information that Alice transfers is associated with multiples *proposals*, which correspond to different channel states. The other party, Bob (or *Approver*), responds with a confirmation, *approval*, of the update, by choosing one of Alice's proposals and making it the next valid state. The advantage of this setting is that (1) the approver always knows what the latest state is as soon as it receives the first round-message and (2) in the case where the approver does not approve the update, there is no need to respond with a message to declare this.

For example, Alice and Bob open a payment channel for Bitcoin (BTC), Ethereum (ETH) and Cardano (ADA). Under-the-hood, they will be opening three bidirectional channels; one for each currency, and after initial setup Alice and Bob should be able to exchange BTC, ETH and ADA in any combination as long as they have sufficient corresponding balance in the channels. More importantly, we want the parties to be able to move the funds allocated to the payment channel to make any type of cross-chain transaction. For example, if Alice has in the payment channel 2 Bitcoin, she should be able to exchange (for example) 1 Bitcoin for Ethereum and 1 Bitcoin for Algorand, or exchange the 2 bitcoins entirely for Ethereum. In a nutshell, the same coin can be used for exchanges across all different chains.

Our Contribution in More Detail. Our offline payment construction is a *bi-directional*, *multi-proposal*, *multi-asset* payment channel in a *proposer/approver* setting, that allows the proposer to make multiple proposals and the approver confirms one of those as valid. The protocol requires minimal scripting requirements from the underlying ledgers (1) transaction authorization (digital signatures), and (2) relative time locks (Check Sequence Verify or CSV). This makes our solution compatible with a variety of blockchains (including Bitcoin).

To elaborate with an example, Alice and Bob can open a channel with, say 3, cryptocurrencies. Say these currencies are Bitcoin (BTC), Ethereum (ETH) and Algorand (ALGO), and their initial balances

(state 1) are `BTC: (Alice: 10, Bob: 0), ETH: (Alice: 10, Bob: 0), ALGO: (Alice: 0, Bob: 100)`. Under the hood, Alice and Bob have 3 bi-directional channels open (one for each cryptocurrency).

They can exchange their assets across any combination of chains. For example, if Alice wants to buy 40 ALGO for 1 BTC and 3 ETH or 80 ALGO for 3 BTC, she *propose*s to update channel balances (to state 2) as follows: `BTC: (Alice: 9, Bob: 1), ETH: (Alice: 7, Bob: 3), ALGO: (Alice: 40, Bob: 60)` or `BTC: (Alice: 7, Bob: 3), ETH: (Alice: 10, Bob: 0), ALGO: (Alice: 80, Bob: 20)`. If Bob *approve*s either proposal, the channel state will move from the state 1 to the state 2. The strength of our construction is that Bob (the approver) gets to decide the final state (the state the channel will close at).

This property is crucial in the settings where parties want to perform high frequency trading. In particular, in the context of decentralized exchanges that securely perform asset swaps across heterogeneous blockchain networks. Indeed, our construction offers a promising primitive for designing cross-chain market-making systems, especially those targeting high-frequency trading scenarios where safeguarding assets across different ledgers is critical.

For example, consider the Market Maker (MM)/Trader [Das05; DMI08; Cia+22; ZQG21; Uni18; Cur20] application. In this, there is an entity called Market Maker that is willing to trade with any trader. The price at which commodities are sold or bought is decided by an automated process based on the amount and the type of currency the traders buy and on the amounts of commodities available to the Market Maker. Our payment channel is perfectly suitable for this context. Indeed, in this setting, we have that: (1) only the traders (proposer) will send a request to trades, proposing to the market maker what currencies and what amounts the traders wants to buy (i.e., the traders propose multiple options to update the channel's state), (2) the MM (approver) can decide what proposal to pick, and know immediately (and decide) whether or not the channel will move to the next state. This is crucial because MM will elaborate the trades sequentially, and before computing the buy/selling prices for the next trade, it must be sure that all the prior trades will actually be executed (i.e., an actual exchange of commodities will occur even if there are corrupted traders).

This setting has been considered in [Cia+22], where off-chain price discovery is enabled via the use of an automated MM that executes front-running–resilient atomic swaps using Ethereum as a settlement layer. Our protocol can be extended to enable similar guarantees across multiple chains.

We implement our protocol, with precise details provided in Section 4, and benchmark it, with details on the experimental setup and computations given in Section 4.1 and in Section 4.2, respectively. In addition, in Table 1 we present the cumulative costs in USD ($) incurred from the opening to the closing of the channel, reflecting the total transaction fees paid by both Alice and Bob over the entire channel lifecycle. Overall, the results indicate that our construction achieves cross-chain atomicity with low operational costs: for both Bitcoin and Ethereum, costs increase only modestly as the number of proposals grows, while for Algorand fees remain negligible.

1.1 Related Works and Comparison

The unidirectional channels [Spi14; Tod14] have simpler constructions because money flows in one direction (from Alice to Bob). Therefore, Bob has no incentive to cheat. However, the money flow in one direction also limits their practical usefulness. The DMC [DW15] construction builds a bidirectional payment channel over 2 uni-directional channels that complement each other along with a *reset* operation that may be run when a party's funds are depleted in the underlying uni-directional channel where he is a sender to readjust his balance from the other underlying uni-directional channel where he is a receiver. Due to the nature of its construction, it does not support an unrestricted number of off-chain payments (as it needs to readjust occasionally, an operation that involves the Blockchain), and the number of transactions to close the channel is higher than any other construction in our comparison. Lightning Network [PD16] is the most widely deployed off-chain payment mechanism, but it does not provide formal security proofs and only briefly mentions cross-chain payments as a potential use case, without focusing on cross-chain atomic swaps. Eltoo [DRO18] and Daric [Mir+22a] both require implementation of opcodes not currently available in Bitcoin, and are therefore offer limited practical utility. Generalized Channels [Aum+21a] are also Bitcoin-compatible, but they are not cross-chain. IvyAPC [Li+25] extends generalized payment channels by introducing auditable

N	Bitcoin		Ethereum	
	Bob init	**Alice init**	**Bob init**	**Alice init**
1	2.54$	2.70$	2.33$	2.45$
2	2.59$	2.74$	2.69$	2.79$
4	2.77$	2.97$	3.29$	3.41$
8	3.14$	3.43$	4.46$	4.57$
16	3.88$	4.37$	6.74$	6.87$

Table 1: At the time of writing, for Bitcoin, the fee rate was 4.0 sat/vB (source: `https://btc.network/estimate`, accessed on 2025-05-16) for inclusion within 6 blocks, and the BTC/USD exchange rate was $103,747.15 (source: CoinMarketCap, accessed on 2025-05-16). For Ethereum, the ETH/USD exchange rate was $2,588.56 (source: CoinMarketCap, accessed on 2025-05-16), and the gas price was 1.72 GWEI. The column `Alice init` indicates that Alice initiated the closing transactions, while `Bob init` denotes that Bob did. The parameter N corresponds to the number of proposals that Alice sends to Bob in order to advance the channel to a new state.

structures that allow an external auditor to verify the completeness, order, and authenticity of off-chain transactions, even in the presence of colluding parties. Compared to Generalized Channels, our construction is limited to simple direct payments and does not support the execution of arbitrary two-party computations off-chain. However, for direct payments, our approach offers reduced communication rounds and is naturally suited for cross-chain setups. Another recent work is Sleepy Channels [Aum+22], which removes the requirement for watchtowers (or Alice and Bob staying online) by using *absolute* rather than *relative* time-locks. Switching to absolute time-locks have the trade-off that the channel's lifetime can no longer be unrestricted. Indeed, the channel must be closed before the absolute time-lock expires. To overcome this issue, such constructions tend to use very large values for absolute time-locks. However, now unresponsive counterparty can effectively keep the channel open till the time-lock expires. The sleepy channel proposes an additional collateral locked in the channel to incentivize counterparty confirmation. This is undesirable since both parties have to lock more coins than they intend to transact on. Moreover, while *absolute* time-locks do not require parties to actively monitor the blockchain through watchtowers, *relative* time-locks impose such a requirement—creating the necessity for continuous monitoring. While our work can trivially adapt the ideas of Sleepy Channels (use absolute time locks, lock additional collateral for fast confirmation), we find the trade-off (restricted channel lifetime, large time-locks, and locking of additional coins) undesirable. In a nutshell, we are the only payment channel construction, to our knowledge, that provides support for multiple asset-types. Moreover, our *update* protocol is the cheapest in terms of round complexity (only two rounds).

DLSAG [Mor+20], and PayMo [Thy+20] are uni-directional payment channels tailored specifically to the Monero blockchain. Z-Channel [Zha+18] is tailored specifically to Zerocash. None of these works is bi-directional which limits practical utility. AuxChannel [Sui+22] provides a bidirectional channel on Monero by realizing a Key Escrow Service (KES) on a second, with turing-complete smart contracts, blockchain (e.g., Ethereum). Our construction is not compatible with Monero, and for Zerocash we do not support the stealth address feature, only the normal address type. We present a comprehensive comparison with several payment channel mechanisms in Table 2.

The concept of payment channel can be taken further to tackle the following scenario. Alice has a payment channel with Bob and Bob has a payment channel with Carol. Alice and Carol do not have a direct payment channel between them, but they can use Bob as an intermediary to make payments to each other. Indeed, any two parties that have a path to each other, even if it is through a number of intermediaries, can make offline payments to each other. This is essentially the idea underlying *payment channel networks (PCNs)* [PD16; Net18; Aum+21c; AAM22; Tiw+22; DFH18; Ava+22; GM17; AST24; Ava+23] and *payment channel hubs (PCHs)* [Hei+17; TMSM21; Gla+22; Ge+23; Qin+23; MS+25; Li+24]. The difference between PCNs and PCHs is network topology; whereas in a PCN different payments may be routed through different paths,

a PCH has a hub-and-spoke structure where any two parties send/receive payments through the central hub. A closely related concept is a *virtual channel* [Dzi+19b; Aum+21b]. Continuing the Alice-Bob-Carol example above, a virtual channel has the advantage that it allows Alice and Carol to send payments to each other without interaction from Bob. Bob only needs to be available during the initial setup/opening of the virtual channel. X-Transfer [Aum+25] is the first secure, scalable, and fully off-chain protocol that enables cross-PCN transfers, even across different blockchains. However those constuctions, are limited to single-currency settings, with a focus on liquidity optimization and throughput. In contrast, our work targets cross-chain, multi-asset exchange, enabling atomic swaps of assets across heterogenous blockchains. We do not compare against these constructions, as their goals differ significantly from ours. While they also build on top of payment channels, their focus is on scalability within a single blockchain ecosystem (e.g., routing, hub efficiency, or virtualized channels), whereas our work targets cross-chain atomic swaps, making the objectives fundamentally incompatible. A much richer construction, often called *State Channel* [Mil+19; DFH18; Lia+22; Dzi+19a; Cha+21a; Cha+21b], leverages a Turing-complete scripting language (e.g., Ethereum's Solidity) and offers sophisticated use-cases to be realized off-chain. However, since such constructions rely on advanced smart contract capabilities that are not available in Bitcoin, whereas in our work, we focus on Bitcoin compatibility.

Table 2: Comparison of Different Payment Channel Schemes. DS is Digital Signatures, CLTV (Check Lock Time Verify) indicates *time locks*, CSV (Check Sequence Verify) indicates *relative time locks*. DLSAG is *Dual-Key* LSAG(Linkable Spontaneous Anonymous Group Signature), LRS is *Linkable Ring Signatures.* "No Expiry" denotes channels with either unrestricted lifetime (via relative time-locks) or fixed lifetime (via absolute time-locks).

Scheme	Initiator	No Expiry	Setup Rounds	Update Rounds	Bidirectional	Cross-chain	Script Requirements
Micro-payments [Spi14]	N/A	✗	2^*	2^*	✗	✗	DS
Payment Channels [Tod14]	N/A	✗	3^*	2^*	✗	✗	DS + CLTV
DMC[4] [DW15]	Either	✗	4^*	$\{2,6\}^{*,1}$	✓[2]	✗	DS + CLTV
Lightning [PD16]	Either	✓	4^*	3^*	✓	✗	DS + CSV
Eltoo [DRO18]	Either	✓	4^*	4^*	✓	✗	DS + CSV+ SIGHASH_NOINPUT[3] + Seq Number
Generalized [Aum+21a]	Either	✓	4	6	✓	✗	DS + CSV
Sleepy Channels [Aum+22]	Either	✗	$2+\Pi_{\mathsf{JKGen}}$	$5+\Pi_{\mathsf{Sign}}$	✓	✗	DS + CLTV
Daric [Mir+22a]	Either	✓	4	6	✓	✗	DS + CSV + SIGHASH_ANYPREVOUT[3]+ Seq Number
DLSAG[5,†] [Mor+20]	N/A	✗	$1+\Pi_{\mathsf{DLSAGKeyGen}}$	$\Pi_{\mathsf{2of2RSSign}}$	✗	✗	DLSAG + CLTV
PayMo[†] [Thy+20]	N/A	✗	$3+\Pi_{\mathsf{KTGen}}+\Pi_{\mathsf{JSpend}}$	$3+\Pi_{\mathsf{JSpend}}$	✗	✗	Monero LRS + CLTV
Z-Channel[6] [Zha+18]	N/A	✗	6	3	✗	✗	CLTV + multisigs + Zerocash DAP
This Work	Designated	✓	5	2	✓	✓	DS + CSV

∗ this is our best guess, the work does not provide formal protocol specification.
† DLSAG and PayMo are specific to Monero.
1 6 rounds if a *Reset* operation needs to be performed.
2 implemented via two uni-directional payment channels.
3 SIGHASH_ANYPREVOUT was previously known as SIGHASH_NOINPUT. At the time of this writing, Bitcoin has not implemented it.
4 number of off-chain payments is limited.
5 requires hard fork on Monero.
6 Z-Channel is specific to Zerocash. Moreover it requires multisigs and time locks, these are not provided by Zerocash.
7 requires a second blockchain that supports smart-contracts.

Industry Interoperability Frameworks. Beyond academic constructions, several industry-driven initiatives aim to address blockchain interoperability. Espresso Systems [Esp23] proposes a decentralized shared sequencer for rollups, establishing a global ordering of transactions across participating rollups. This ensures, for example, that if a user submits transaction TxA to Rollup A and transaction TxB to Rollup B, all rollups will agree that TxA precedes TxB in the global log. However, the framework does not guarantee that the transactions succeed, nor that they execute atomically across rollups. Polygon's AggLayer [Pol24]

connects zk-rollups by aggregating their validity proofs and publishing them to Ethereum. Instead of each rollup maintaining its own independent bridge, AggLayer provides a unified bridge contract on Ethereum that verifies aggregated proofs and coordinates cross-chain transfers. This architecture allows assets and messages to move consistently between participating rollups, enabling liquidity to be shared across chains and making user interactions feel closer to a single, seamless multi-rollup environment. Chainlink's Cross-Chain Interoperability Protocol (CCIP) [Cha24] enables cross-chain messaging and token transfers through an oracle network, facilitating application-level interoperability but inheriting off-chain trust assumptions. SafeNet [Tha24] and similar hub-based frameworks achieve interoperability via custodial intermediaries, making it easier for applications to integrate by connecting only to the hub rather than implementing complex cross-chain protocols, but at the cost of decentralization and with the added trust dependency on the hub operator. While these approaches advance scalability, liquidity sharing, and ecosystem-level connectivity, they are not designed to natively support two-party atomic swaps. Rather, they serve as infrastructure layers upon which higher-level applications may build such functionality. Our work therefore complements these frameworks by focusing on trust-minimized, Bitcoin-compatible atomic swaps with strong two-party fairness guarantees.

1.2 Technical Overview

Informally, the protocol works as follows. Assume that there are two blockchains involved. Alice and Bob open two standard payment channels, one of the first chain B_1 and one of the second chain B_2. Alice and Bob open a bi-directional (non-cross-chain) payment channel PC_1 on B_1 and another bi-directional channel PC_2 on B_2. A state update can be thought of as two independent updates that happen respectively on PC_1 and PC_2. It is however critical to make sure that both updates happen *atomically*. To enforce this, at every state we associated hash values, for which Alice and Bob need to reveal the corresponding pre-image (the revocation keys) to acknowledge the fact that they want to move to a next state and make the previous state invalid.

For example, when Bob receives an updated request from Alice, he needs to disclose his revocation keys (hash pre-images) on both chains B_1 and B_2. Consider now a malicious Bob that attempts to update the state of only one channel PC_1, and close the channel at this new state (but he closes the channel at the previous state on PC_2). To do that, Bob needs to issue the revocation keys for the previous state. In our protocol we make sure that this revocation keys can be used by Alice on PC_2, to prove that Bob tried to advance the state of the channel PC_1, without advancing the state of PC_2. Unless Bob counters Alice's complain on PC_2 by properly closing the channel at the updated state, then all the amounts locked in both channels will be transferred to Alice. This is how we enforce that Bob updates the states on both chains.

As mentioned, our approach works even when more than two chains are considered. One could think that extending the above approach to this setting is trivial by applying the above technique *pairwise*. Hence, creating a cross-chain payment channel between every pair of chains. This straw-man approach has a clear massive disadvantage. Indeed the coins locked in one of the channel, cannot be used in other channels. Hence, if for example we want to trade 1 Bitcoin for Ethereum and Algorand, we need to lock 1 Bitcoin in all the cross-chain channels (Bitcoin-Ethereum, Bitcoin-Algorand).

We instead generalize the solution proposed above, to avoid this problem, and to enable Alice to make multiple proposals, forcing Bob to accept only one of them while discarding the others atomically. Rather than creating a separate cross-chain channel for every pair of ledgers, we open only one channel per chain and define the update mechanism jointly across all chains. Each new state therefore reflects balances on every ledger at once, and revocation keys are tied consistently across chains, so that a deviation on one chain can be punished on the others. This ensures that the same coin locked in its native channel can be reused across multiple cross-chain swaps, eliminating the need to duplicate collateral across pairwise channels. Achieving this while relying only on Bitcoin-like scripting capabilities represents a non-trivial challenge, which we address in the technical part of the paper.

2 Preliminaries

We denote $x := y$ the operation of assigning y to x. For an algorithm $\mathcal{A}$, denote $y \leftarrow \mathcal{A}(x)$ the operation of running $\mathcal{A}$ with input x and assigning the result to y. $[elem]_{\mathsf{N}}$ stands for a list of up to N elements. $(elem1, elem2, ...)$ stands for a tuple of elements. $a \preceq b$ denotes that a is a prefix of b. We use $\mathcal{H}$ to define a globally available hash function. We consider time as divided into discrete units called *rounds* or *time-slots*. For simplicity we assume the parties are synchronized—that is, honest parties agree on which round it is.

2.1 Signatures

Definition 1 (Signatures)**.** *A signature scheme* SIG *consists of the following three algorithms* $(\mathtt{Gen}, \mathsf{Sign}, \mathsf{Ver})$.

- $(pk, sk) \leftarrow \mathtt{Gen}(1^\lambda)$. *The key generation algorithm takes as input the security parameter* λ*, it outputs a public key* pk *and a secret key* sk.
- $\sigma \leftarrow \mathsf{Sign}(sk, m)$. *The signing algorithm takes as input* sk *and a message* m*, it outputs a signature* σ.
- $0/1 \leftarrow \mathsf{Ver}(pk, m, \sigma)$. *The verification algorithm takes as input* pk*,* m*, and* σ*, it outputs a bit* b.

Correctness*. For any* $(pk, sk) \leftarrow \mathtt{Gen}(1^\lambda)$*, any* m *and* $\sigma \leftarrow \mathsf{Sign}(sk, m)$*, it holds that* $\mathsf{Ver}(pk, m, \sigma) = 1$.

Definition 2 (Unforgeability of Signatures)**.** *A signature scheme* SIG *is unforgeable under chosen message attacks (UF-CMA secure), if for any PPT adversary* $\mathcal{A}$*, the advantage*

$$\mathcal{A}^{uf}_{\mathsf{SIG},\mathcal{A}}(\lambda) := \Pr\left[\mathsf{Exp}^{uf}_{\mathsf{SIG},\mathcal{A}}(\lambda) = 1\right]$$

is negligible over λ*, where the experiment* $\mathsf{Exp}^{uf}_{\mathsf{SIG},\mathcal{A}}(\lambda)$ *is defined in Fig. 1.*

$\mathsf{Exp}^{uf}_{\mathsf{SIG},\mathcal{A}}(\lambda)$:	$\textsc{Sign}(m)$:
$(pk, sk) \leftarrow \mathtt{Gen}(1^\lambda)$; $\mathcal{S} := \varnothing$	$\sigma \leftarrow \mathsf{Sign}(sk, m)$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\textsc{Sign}(\cdot)}(pk)$	$\mathcal{S} := \mathcal{S} \cup \{m\}$
	Return σ
Return $((m^* \notin \mathcal{S}) \wedge (\mathsf{Ver}(pk, m^*, \sigma^*)))$	

Figure 1: The UF-CMA security experiment of signature scheme SIG.

2.2 Blockchain

Definition 3 (Blockchain Ledger)**.** *A blockchain ledger* $\mathcal{L}$ *is a distributed protocol executed by a set of parties. It maintains an append-only data structure that records a totally ordered sequence of transactions. Each party* P *at time slot* t *holds a local view of the ledger state, denoted by* $\mathcal{L}_P[t]$.

The ledger exposes the following interface:

- $\mathtt{Submit}(tx)$*: Allows a party to submit a transaction* tx *for inclusion in the ledger.*
- $\mathtt{Read}()$*: Returns the current view* $\mathcal{L}_P[t] = [tx_1, tx_2, \ldots, tx_k]$*, representing the stable prefix of the ledger observed by party* P *at time slot* t.

A ledger protocol $\mathcal{L}$ is secure if it enjoys the following properties (see [Cia+20]):

Consistency. For any two honest parties P_1, P_2 and two time slots $t_1 \leq t_2$, it holds that $\mathcal{L}_{P_1}[t_1] \preceq \check{\mathcal{L}}_{P_2}[t_2]$

Liveness. If any honest party in the system attempts to include a transaction Tx, then, at any slot t, after s slots (called the liveness parameter) , any honest party P, if queried, will report $\mathsf{Tx} \in \mathcal{L}_P[t]$.

Common Prefix (CP) with parameter $k \in \mathbb{N}$, states that for any pair of honest parties P_1, P_2 at rounds $r_1 \leq r_2$ respectively, it holds that: $\mathcal{L}_{P_1}[r_1] \preceq \check{\mathcal{L}}_{P_2}[r_2]$.

Chain Growth (CG) with parameters $\tau \in (0, 1]$ **and** $s \in \mathbb{N}$. Consider the chain C adopted by an honest party at the onset of a slot and any portion of C spanning s prior slots; then the number of blocks appearing in this portion of the chain is at least τs blocks.

2.3 UTXO model.

In this paper we consider the Unspent Transaction Output (UTXO) model. Transactions consist of a mapping from inputs to outputs. Each output is associated with a predicate that must be satisfied for the output to be later consumed, and each input is accompanied by the arguments required to satisfy the corresponding predicate. A transaction can thus be described as

$$\mathsf{Tx}_k := [(inp_i^1, \mathsf{arg}_i^1), (inp_i^2, \mathsf{arg}_i^2), (inp_j^2, \mathsf{arg}_j^2), ..] \rightarrow [(out_k^1, \mathsf{pred}_k^1), (out_k^2, \mathsf{pred}_k^2), ..]$$

where i, j represent the indexes of previously issued transactions, and k denotes the identifier of the new transaction. Each pred_k^ℓ with $\ell \in \mathbb{N}$ is a predicate that must evaluate to `TRUE` for the corresponding output out_k^ℓ to be spendable, and each arg_i^m is a set of arguments meant to satisfy the predicate guarding inp_i^m.

The ledgers we consider must support validation of the following three predicate literals, which are essential for our transaction semantics:

1. $\mathsf{HashLock}(S, r) := \mathsf{TRUE} \iff H(r) = S$. This predicate evaluates to TRUE if a valid preimage r is provided such that it hashes to a given commitment S.
2. $\mathsf{RelativeTimeLock}(t, \mathsf{ref}) := \mathsf{TRUE} \iff \mathsf{blockTime}_{\mathsf{current}} \geq \mathsf{blockTime}_{\mathsf{ref}} + t$. This predicate ensures that an input can only be spent if the consuming transaction is included at least t units (e.g., blocks) after the referenced input's inclusion.
3. $\mathsf{SigVerify}(pk, \sigma, inp_i^j) := \mathsf{TRUE} \iff \mathsf{Verify}(\mathtt{pk}, inp_i^j, \sigma) = \mathsf{TRUE}$. This predicate is satisfied if a valid digital signature σ under public key pk is provided for the UTXO inp_i^j (typically derived from the transaction context).

In the Bitcoin scripting language (`Script`) [Bit25], HashLocks can be created using `OP_HASH160` followed by `OP_EQUALVERIFY`. RelativeTimeLocks are enforced with `OP_CHECKSEQUENCEVERIFY`, while signature verification is achieved using `OP_CHECKSIGVERIFY`. In addition to these primitives, we require minimal control flow capabilities. Specifically, the scripting interpreter (or compiler) must support branching logic equivalent to `OP_IF`, `OP_ELSE`, and `OP_ENDIF`.

3 Cross-chain Lightning Trades

Our first contribution is the formalization of a cross-chain payment channels that captures all the feature we mentioned in the introductory part of the paper. Below we provide our formal definition.

Definition 4. *A two-party multi-asset multi-proposals bi-directional payment channel protocol Π_{BPC} with designated proposer $\mathsf{P1}$ consists of the following 5 algorithms (Setup, Update, UpdateProceed, RequestClose, Close):*

- $(\mathsf{st}, \mathsf{priv}_1, \mathsf{priv}_2) \leftarrow$ *Setup*$(\mathsf{B}, \mathsf{F}_{\mathsf{P1}}, \mathsf{F}_{\mathsf{P2}}, \Delta_{\mathsf{B}}, \mathsf{N})$*: The Setup is a two-party protocol that initializes the payment channel by locking funds on the blockchains enumerated in the list of participating blockchains* B*. Specifically,* F_{P1} *and* F_{P2} *denote vectors that hold the amount of coins that* $\mathsf{P1}$ *and* $\mathsf{P2}$*, respectively lock on each blockchain* $b \in \mathsf{B}$*. Transactions are submitted on-chain with respect to the timeout parameter* Δ_{B}*, which defines the response window for either party. The parameter* N *sets an upper bound on the number of proposals* $\mathsf{P1}$ *can create for each state. The output consists of the initial state* st *and the private outputs* priv_1*,* priv_2*, returned respectively to* $\mathsf{P1}$ *and* $\mathsf{P2}$*.*
- $(\mathsf{proposals}, \mathsf{priv}_1') \leftarrow$ *Update*$(\mathsf{st}, [(\mathsf{F}_{\mathsf{P1}}', \mathsf{F}_{\mathsf{P2}}')]_{\mathsf{N}}, \mathsf{priv}_1)$*: The Update algorithm is executed by party* $\mathsf{P1}$ *and takes as input the channel state* st*, the private input* priv_1*, and a list of proposed new mappings* $\mathsf{F}_{\mathsf{P1}}'$ *and* $\mathsf{F}_{\mathsf{P2}}'$ *that represent the new vectors that store balances of* $\mathsf{P1}$ *and* $\mathsf{P2}$ *for each participating blockchain for each proposed state. It outputs a list of proposed states named* $\mathsf{proposals}$ *that is sent to* $\mathsf{P2}$ *and the new private state of the proposer* priv_1*.*
- $(\mathsf{st}', \mathsf{priv}_2') \leftarrow$ *UpdateProceed*$(\mathsf{st}, \mathsf{proposals}, \mathsf{index}, \mathsf{priv}_2)$*: The UpdateProceed algorithm is executed by party* $\mathsf{P2}$ *and takes as input* st*, proposals, an index and the approver's private state* priv_2*. It returns the updated channel state* st' *(which reflects proposals[index]) that is sent to* $\mathsf{P1}$ *and the new private output of the approver* priv_2*.*

- *ReqClose*$(\mathsf{B}, \mathsf{st})$*: The ReqClose algorithm takes as input the list of participating blockchains* B *and* st*. It initializes blockchain transactions to initiate settling the channel on blockchains indexed by* B*.*
- *Close*$(\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B})$*: The Close algorithm is a two party protocol that takes as input the private states* priv_1*,* priv_2*, the state* st *and* B*. It moves the funds back to the users according to* st*.*

Correctness. Consider the following steps in an execution of the protocol Π_{BPC} between two honest parties P1 and P2.

1. **Setup:** $(\mathsf{st}, \mathsf{priv}_1, \mathsf{priv}_2) \leftarrow \mathrm{Setup}(\mathsf{B}, \mathsf{F}_{\mathsf{P1}}, \mathsf{F}_{\mathsf{P2}}, \Delta_{\mathsf{B}}, \mathsf{N})$
2. **Update by P1:** $(\mathsf{proposals}, \mathsf{priv}_1') \leftarrow \mathrm{Update}(\mathsf{st}, [(\mathsf{F}_{\mathsf{P1}}', \mathsf{F}_{\mathsf{P2}}')]_{\mathsf{N}}, \mathsf{priv}_1)$
3. **UpdateProceed by P2:**$(\mathsf{st}', \mathsf{priv}_2') \leftarrow \mathrm{UpdateProceed}(\mathsf{st}, \mathsf{proposals}, \mathsf{index}, \mathsf{priv}_2)$
4. **Locally, P1 sets** $\mathsf{priv}_1 := \mathsf{priv}_1'$ **and P2 sets** $\mathsf{priv}_2 := \mathsf{priv}_2'$**.**
   Both parties set $\mathsf{st} := \mathsf{st}'$**.**
   REPEAT steps 2–3–4 m **times.**
5. **Close request by either Party:** $\mathrm{ReqClose}(\mathsf{B}, \mathsf{st})$
6. **Channel finalization:** $\mathrm{Close}(\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B})$

We say that Π_{BPC} enjoys correctness if it holds that for any $m, \mathsf{B}, \mathsf{F}_{\mathsf{P1}}, \mathsf{F}_{\mathsf{P2}}, \Delta_{\mathsf{B}}$, N such that $\mathrm{Setup}(\mathsf{B}, \mathsf{F}_{\mathsf{P1}}, \mathsf{F}_{\mathsf{P2}}, \Delta_{\mathsf{B}}, \mathsf{N})$ and for any $\mathsf{F}_{\mathsf{P1}}', \mathsf{F}_{\mathsf{P2}}'$ such that $\mathsf{F}_{\mathsf{P1}}[b] + \mathsf{F}_{\mathsf{P2}}[b] = \mathsf{F}_{\mathsf{P1}}'[b] + \mathsf{F}_{\mathsf{P2}}'[b]\ \forall b \in \mathsf{B}$ then if the channel is closed via $\mathrm{Close}(\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B})$, where st is the state resulting from the last successful invocation of `UpdateProceed` involving $\mathsf{F}_{\mathsf{P1}}', \mathsf{F}_{\mathsf{P2}}'$, then the balances $\mathsf{F}_{\mathsf{P1}}'[b]$ and $\mathsf{F}_{\mathsf{P2}}'[b]$ are returned on-chain to the accounts of P1 and P2, respectively, for every $b \in \mathsf{B}$. That is,

$$\Pr\left[\mathtt{Close}(\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B})\right] \geq 1 - \mathsf{negl}(\lambda).$$

A visual summary of the steps is depicted in Figure 2.

Security Properties. We define the security properties of Π_{BPC} through three notions: *responsive closure*, *proposer protection* and *approver protection*, starting with our adversarial model.

Adversarial Model. We model security against a malicious, static adversary in a two-party setting. The adversary corrupts either the proposer or the approver at the start of the protocol and may arbitrarily deviate from the prescribed execution.

Responsive closure. Responsive closure ensures that any honest party holding a valid state st can unilaterally trigger on-chain settlement, and that the counter-party is granted a bounded time window, denoted by Δ_B, to finalize the settlement. This guarantees that the honest party can ultimately retrieve their rightful balance, regardless of the adversary's behavior.

Definition 5. *(Responsive closure) A two-party multi-asset multi-proposals bidirectional payment channel protocol* Π_{BPC} *with designated proposer achieves responsive closure, if for any* PPT *adversary* $\mathcal{A}$ *there exists a negligible function* negl *such that the probability*

$$\Pr\left[\mathsf{Exp}^{\mathcal{A}}_{RC,(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}(\lambda) = 1\right] \leq negl(\lambda)$$

where the experiment $\mathsf{Exp}^{\mathcal{A}}_{RC,(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}$ *is described in Figure 3.*

Proposer protection. Proposer protection ensures the safety of an honest proposer. It guarantees that even if the approver is controlled by an adversary, the adversary cannot claim more funds than those specified in the final closing state st. This state must either correspond to the last result of UpdateProceed, or, if the adversary has not executed UpdateProceed, to a state proposed in the most recent invocation of Update.

Definition 6. *(Proposer protection) A two-party multi-asset multi-proposals bidirectional payment channel protocol* Π_{BPC} *with designated proposer achieves proposer protection, if for any* PPT *adversary* $\mathcal{A}$ *there exists a negligible function* negl *such that the probability*

$$\Pr\left[\mathsf{Exp}^{\mathcal{A}}_{PP,(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}(\lambda) = 1\right] \leq negl(\lambda)$$

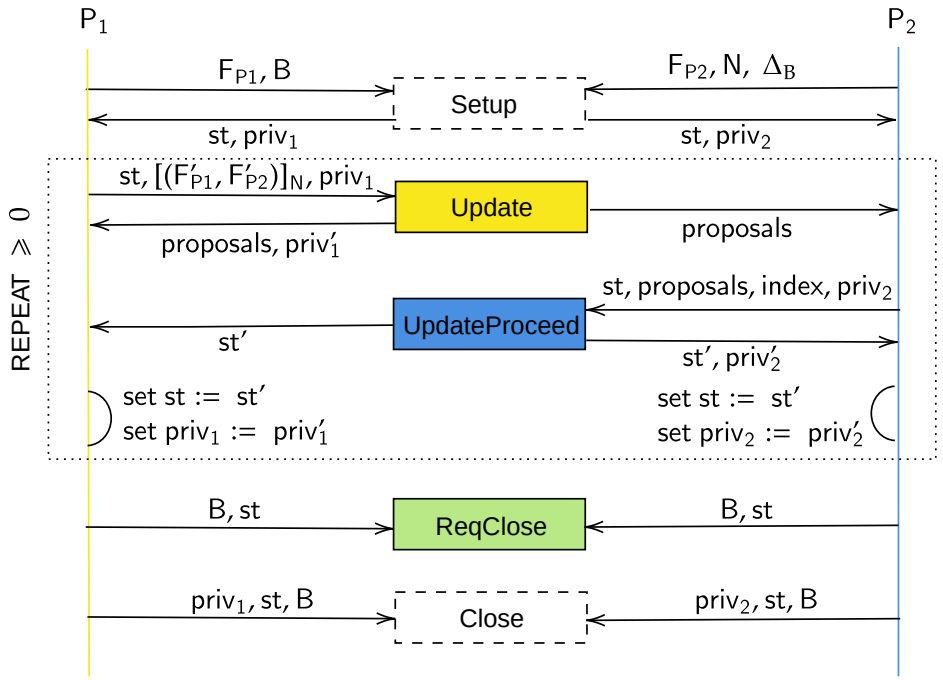

Figure 2: High-level overview of the parties' interaction. Yellow background indicates operations run locally by party P1, and blue background indicates operations run locally by P2. Rectangles with dashed borders represent two-party protocols executed jointly. Green background denotes operations that either party can perform. Rectangles with white or green backgrounds initiate blockchain transactions upon invocation; all others are off-chain, local computations. The arrows point to the direction of information flow.

where the experiment $\mathsf{Exp}^{\mathcal{A}}_{PP,(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}$ *is described in Figure 4, the probability is taken over all random choices.*

Approver protection. Approver protection ensures the safety of an honest approver. It guarantees that even if the proposer is controlled by an adversary, the adversary cannot claim more funds than those specified in the final closing state st. This state must be the last result of UpdateProceed.

Definition 7. *(Approver protection) A two-party multi-asset multi-proposals bidirectional payment channel protocol* Π_{BPC} *with designated proposer achieves approver protection, if for any* PPT *adversary* $\mathcal{A}$ *there exists a negligible function* negl *such that the probability*

$$\Pr\left[\mathsf{Exp}^{\mathcal{A}}_{AP,(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)=1\right] \leq negl(\lambda)$$

where the experiment $\mathsf{Exp}^{\mathcal{A}}_{AP,(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}$ *is described in Figure 5, the probability is taken over all random choices.*

3.1 Protocol Overview

At a high level, in the first round, Alice proposes to move the channel to state i by sending a bunch of proposals, and in the second round, Bob confirms that the channel has moved to state i by making one of those proposals the current state. Note that if Alice tries to close the channel after the first round, Bob can

$\mathsf{Exp}^{\mathcal{A}}_{RC,(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}$

1. The adversary $\mathcal{A}$ statically corrupts either P1 or P2.
2. Both parties jointly execute the Setup. The adversary can abort.
3. The adversary may interact with the honest party arbitrarily many times. If P1 is corrupted, the adversary can choose to execute `Update` with arguments of its choice or abort. If P2 is corrupted, the adversary can similarly choose to execute `UpdateProceed` with arguments of its choice or abort.
4. Eventually $\mathcal{A}$ finishes the game and closes the channel.
5. The adversary wins the responsive closure game if the honest party is unable to successfully complete on-chain settlement and retrieve, on any $\mathsf{b} \in \mathsf{B}$, an amount of funds at least equal to that specified in the most recent mutually agreed state st. Specifically, st refers to the output of `Setup` if `UpdateProceed` has never been invoked, or the output of the last invocation of `UpdateProceed` otherwise.
6. If $\mathcal{A}$ wins the output of the experiment is 1 else 0.

Figure 3: Security game for responsive closure

$\mathsf{Exp}^{\mathcal{A}}_{PP,(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}(\lambda)$

1. The adversary $\mathcal{A}$ statically corrupts P2.
2. Both parties jointly execute the Setup. The adversary can abort.
3. The adversary can interact with P1 by invoking `UpdateProceed` with arguments of his choice or abort.
4. Eventually $\mathcal{A}$ finishes the game.
5. The adversary wins the proposer protection game if:
   - The channel settles in different states across different blockchains; or
   - The closing state st is not equal to any state proposed by P1 in the last invocation of Update, in case the adversary did not execute `UpdateProceed` (abort); or if `UpdateProceed` was executed, st differs from the last output of `UpdateProceed`.
6. If $\mathcal{A}$ wins the output of the experiment is 1 else 0.

Figure 4: Security game for proposer protection

settle at either state $i-1$ or any proposal for state i that Alice made (same state/proposal on all blockchains, otherwise he is punished). Indeed, Alice should not be able to force Bob to move to state i unless (Bob decides to do so). However, we do ask Bob to reveal the appropriate revocation keys as part of the channel settlement. This prevents Bob from cheating by settling on state $i-1$ on the one blockchain and on i on the other. After the second round, the channel is confirmed to be at state i i.e., Bob no longer has the option to settle at $i-1$. Below we provide a more detailed description on how the protocol works, and then give a fully formal description of it in the next section.

Notation. We refer to set of all blockchains as $\mathcal{B} := \{B_1, \ldots, B_K\}$ where $K \geq 1$. The upper bound of proposals that Alice can propose in one channel update is denoted by N and $\mathcal{I}_i := \{M_{i,1}, \ldots, M_{i,\mathsf{N}}\}$ represents a set of hashes supplied by Bob that Alice is using to construct proposals for channel state i. In this section it will be useful to use the following syntax when referring to a transaction: $[\mathsf{Tx}_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$ denotes a transaction Tx on blockchain $b \in \mathcal{B}$ for the channel state $i \in \{1, 2, 3, \ldots\}$, associated with hash $M_{i,j} \in \mathcal{I}_i$, signed by both Alice A and Bob B. Note that Tx is a placeholder for the name of the transaction. We will replace Tx with the actual type of transaction (more details will follow).

$\mathsf{Exp}^{\mathcal{A}}_{AP,(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_{\mathsf{B}},\mathsf{N})}(\lambda)$

1. The adversary $\mathcal{A}$ statically corrupts $\mathsf{P1}$.
2. Both parties jointly execute the Setup. The adversary can abort.
3. The adversary can interact with $\mathsf{P2}$ by invoking `Update` with arguments of his choice or abort.
4. Eventually $\mathcal{A}$ finishes the game.
5. The adversary wins the proposer protection game if:
   - The channel settles in different states across different blockchains; or
   - The closing state `st` is not equal to the result of last execution of `UpdateProceed`.
6. If $\mathcal{A}$ wins the output of the experiment is 1 else 0.

Figure 5: Security game for approver protection

We use the circle symbol, $\bullet$, in the subscript for either A or B denotes that the corresponding signature is needed before the transaction can be broadcast. We abuse the notation by denoting as $[\mathsf{Tx}_{(*,i,j)}]$ the following transaction set: $\forall_{b\in\mathcal{B}}[\mathsf{Tx}_{(b,i,j)}]$ i.e., the transactions for some state i that Alice constructed using hash $M_{i,j}$ on all blockchains in consideration. In addition, we refer to the hash from $\mathcal{I}_i$ that is associated with the mutually agreed state as M_{i,a_i} where a_i is the index of the element in the set $\mathcal{I}_i$. The protocol defines revocation keys $r^{\mathsf{B}}_{i,j}$, r^{A}_{i}, for Bob and Alice respectively, where i is the round associated with the keys and $M_{i,j}$ belongs in $\mathcal{I}_i$. Specifically $r^{\mathsf{B}}_{i,j}$, is derived as $M_{i,j} \leftarrow \mathcal{H}(r^{\mathsf{B}}_{i,j})$ where $\mathcal{H}$ is modelled as a random oracle. This guarantees that, with overwhelming probability, the revocation keys are (1) unique, and (2) cannot be predicted by the counterparty (all parties are assumed to be probabilistic polynomial-time).

Alice, reveals r^{A}_{i} only when she agrees that the channel has moved to state $i+1$, otherwise r^{A}_{i} remains secret. Bob reveal all $r^{\mathsf{B}}_{i,j}$ $\forall_{M_{i,j}\in\mathcal{I}_i\setminus\{M_{i,a_i}\}}$ as well as $r^{\mathsf{B}}_{i-1,a_{i-1}}$, when agrees that the channel has moved to state i + 1, otherwise no revocation key needs to be revealed by Bob. The transactions are setup such that if a cheating party p tries to settle/close the channel—by broadcasting a transaction in the set $[\mathsf{Tx}_{(*,i,j)}]$—at some stale state i i.e., after $r^{B}_{i,j}$ or r^{A}_{i} has been revealed, the counter-party can use the revocation key to prove cheating and take the full balance of the channel, thereby punishing the cheating p.

We denote by $\nu^{p}_{b,i,j}$, with $p \in \{\mathsf{A},\mathsf{B}\}$, the balance of the party p in the channel on blockchain b in state i, which represents a proposal constructed with hash $M_{i,j}$. $\nu_{b,i,j}$ refers to the total balance in the channel on blockchain b. Note that, for a given blockchain b, the total balance in the channel remains the same regardless of the state i or $M_{i,j}$, therefore we can safely omit the state i and hash $M_{i,j}$ when discussing the total balance. In our protocol, we consider different Δ-parameters representing the *delays*. These parameters are related to the transaction liveness of the blockchain involved. Concretely, a delay is specified in an appropriate number of blocks for the underlying blockchain. Δ_{A} is the delay for Alice (she needs to wait Δ_{A}, before claiming the entire balance, in case Bob does not respond on-chain), Δ_{B} is the delay for Bob (he needs to wait Δ_{B} before he claims $v^{B}_{b,i,j}$ giving time to Alice to present $r^{B}_{i,j}$,if she knows it), and Δ_b is the delay in settlement on some blockchain b.

Say, $\widehat{b}$ is the slowest blockchain i.e., $\forall b \neq \widehat{b}$ such that $b \in \mathcal{B}$ and $\Delta_{\widehat{b}} \geq \Delta_b$, we require that $\Delta_{\mathsf{B}} \geq \Delta_{\mathsf{A}} + \Delta_{\widehat{b}}$. Note that across different blockchains, the abstraction of these transactions remains consistent. Therefore, for simplicity, we refer to a single blockchain b in the description below, rather than enumerating all blockchains involved in the setup.

Types of transactions we use. Our construction is enabled by the following transactions (we refer to Appendix A for a formal specification of their inputs, outputs, and predicates).

1. **Funding Transaction F_b.** This is the usual funding transaction that PCs use to be instantiated. Alice and Bob may use any appropriate number of inputs to provide their initial balance in the channel (that is $\mathsf{F_{P1}[b]}, \mathsf{F_{P2}[b]}$ respectively). It has only one output ν_b; the full balance of the channel.
2. **Transaction $\mathsf{W}_{(b,i,j)}$.** When Bob wants to close the channel, he first broadcasts $[\mathsf{W}_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$ and

then, within Δ_{A} rounds, broadcasts $[\mathsf{X}_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$. The transaction $\mathsf{W}_{(b,i,j)}$ takes as input the output of F_b and has a single output ν_b. If Bob does not broadcast transaction $\mathsf{X}_{(b,i,j)}$ within Δ_{A} rounds, Alice can claim the output ν_b of transaction W by presenting a transaction that verifies w.r.t her public key $\mathtt{pk}_{\mathsf{A}}$.

3. **Transaction $\mathsf{X}_{(b,i,j)}$.** Broadcasting $[\mathsf{X}_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$ entails that Bob has revealed $(r^{\mathsf{B}}_{i-1,a_{i-1}}, r^{\mathsf{B}}_{i,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}\setminus\{j\}}, r^{\mathsf{B}}_{i+1,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}})$ i.e., revocation keys for states $i-1$, i and $i+1$ respectively. $\mathsf{X}_{(b,i,j)}$ takes the output ν_b of transaction $\mathsf{W}_{(b,i,j)}$ and has outputs $(\nu^{\mathsf{A}}_{b,i,j}, \nu^{\mathsf{B}}_{b,i,j})$—retrieved from $[(\mathsf{F}'_{\mathsf{P1}}[\mathsf{b}], \mathsf{F}'_{\mathsf{P2}}[\mathsf{b}])]_{\mathsf{N}}[j]$—, these correspond to the balance of Alice and Bob, respectively, at state i based on proposal constructed with hash $M_{i,j}$. Alice can immediately claim $\nu^{\mathsf{A}}_{b,i,j}$ using a transaction that verifies w.r.t. $\mathtt{pk}_{\mathsf{A}}$. If she can show preimage of $M_{i,j}$ i.e., $r^{\mathsf{B}}_{i,j}$, she can also claim $\nu^{\mathsf{B}}_{b,i,j}$. However, unless Bob tries to cheat, Alice will not know $r^{\mathsf{B}}_{i,j}$. After Δ_{B} rounds, Bob can claim $\nu^{\mathsf{B}}_{b,i,j}$ by broadcasting a transaction that verifies w.r.t. his public key $\mathtt{pk}_{\mathsf{B}}$.
Note: It may seem counter-intuitive that we have an intermediate transaction i.e., W, between F and X. Why not make X take as input F directly? The reason is bitcoin compatibility. The "smart contracts" in Bitcoin are tied to output validation scripts. The state of these contracts (output validation scripts) is fixed when the transaction (that they are part of) is posted. There is no way to dynamically set it later. Since, we want to force Bob to reveal $(r^{\mathsf{B}}_{i-1,a_{i-1}}, r^{\mathsf{B}}_{i,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}\setminus\{j\}}, r^{\mathsf{B}}_{i+1,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}})$ when he broadcasts X, and since the output validation script must verify that correct keys were revealed. Making X directly spend F means that the output validation script of F must contain hashes of all revocation keys that Bob may ever use i.e., the output validations script must contain $\{h_{r^{\mathsf{B}}_{0,1}}, \ldots, h_{r^{\mathsf{B}}_{i,j}}\}$. This makes the transaction size prohibitively large. Instead, we introduce the intermediate transaction $\mathsf{W}_{(b,i,j)}$ for each state i and proposal created with $M_{i,j}$, its output validation script only needs to contain hashes of the following revocation keys $\{h_{r^{\mathsf{B}}_{i-1,a_{i-1}}}, h_{r^{\mathsf{B}}_{i,k}}\forall_{k\in\{1,\ldots,\mathsf{N}\}\setminus\{j\}}, h_{r^{\mathsf{B}}_{i+1,k}}\forall_{k\in\{1,\ldots,\mathsf{N}\}}\}$ corresponding to state $i-1$, i and state $i+1$.
4. **Transaction $\mathsf{T}_{(b,i,a_i)}$.** When Alice wants to close the channel, she broadcasts $[\mathsf{T}_{(b,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$. It takes as input the output of F_b and has a single output ν_b that can be spent in the following ways;
   - Δ_{A} rounds pass, Alice claims the full balance ν_b using a transaction that verifies w.r.t. her public key $\mathtt{pk}_{\mathsf{A}}$. Note that this case exists to defend against an unresponsive Bob.
   - Bob claims the full balance of the channel by broadcasting r^{A}_i (preimage of $h_{r^{\mathsf{A}}_i}$). This proves that Alice posted a stale transaction $\mathsf{T}_{(b,i,j)}$ i.e., she is cheating.
   - Bob broadcasts one of $[\mathsf{Y}\circ_{(b,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$ or $[\mathsf{Y}+_{(b,i,a_{i+1})}]_{(\mathsf{A},\mathsf{B})}$.
5. **Transactions $\mathsf{Y}\circ_{(b,i,a_i)}$ and $\mathsf{Y}+_{(b,i,a_{i+1})}$.** These transactions take the input ν_b of $\mathsf{T}_{(b,i,a_i)}$ and have output values corresponding to the channel state i and $i+1$ respectively. Those values are denoted as $(\nu^{\mathsf{A}}_{b,i,j}, \nu^{\mathsf{B}}_{b,i,j})$—retrieved from $[(\mathsf{F}'_{\mathsf{P1}}[\mathsf{b}], \mathsf{F}'_{\mathsf{P2}}[\mathsf{b}])]_{\mathsf{N}}[j]$—, and correspond to the balance of Alice and Bob, respectively. They are very similar in structure. Alice's can always immediately claim her balance. Additionally she can also claim Bob's balance if she can present appropriate revocation key to prove Bob's cheating. Bob must always wait Δ_{B} rounds before he can claim his balance. When Bob broadcasts $\mathsf{Y}\circ_{(b,i,a_i)}$ he reveals the tuple $(r^{\mathsf{B}}_{i-1,a_{i-1}}, r^{\mathsf{B}}_{i,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}\setminus\{a_i\}}, r^{\mathsf{B}}_{i+1,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}})$. On the other hand when he broadcasts $\mathsf{Y}+_{(b,i,a_{i+1})}$, he reveals the tuple $(r^{\mathsf{B}}_{i,a_i}, r^{\mathsf{B}}_{i+1,k}\forall_{k\in\{1,\ldots,\mathsf{N}\}\setminus\{a_{i+1}\}})$.

A well-known challenge in the Lightning Network is the requirement that parties (or their watchtowers [ALW20; Mir+21; Ava+18; KNW19; McC+19; Mir+22b]) continuously monitor the Bitcoin blockchain to ensure that their counterparty does not attempt to close the channel on an outdated state. Our construction inherits this requirement but amplifies it in the cross-chain setting: now, all participating blockchains must be monitored, and each settlement attempt must be responded to on every chain where it occurs. This introduces significant overhead compared to single-ledger channels, because each party must (i) keep a local record of the latest confirmed state, (ii) store all revocation keys disclosed so far (for all proposals and states), and (iii) maintain the corresponding set of partially-signed settlement transactions (T, W, X, $\mathsf{Y}\circ$, $\mathsf{Y}+$) for every admissible closure path for every proposal, state, and involved ledger. These artifacts are essential not only for observing but also for timely reacting within the dispute windows defined by the underlying ledgers (Δ_A, Δ_B).

A visual summary[1] of the transaction dependency graph, showing how the transactions are connected, is provided in Figure 6. It also illustrates the full settlement flow, which may appear complex because it encodes every possible closing scenario across multiple chains. To aid intuition, one can think of the diagram as showing that whenever a counterparty triggers a transaction on one ledger (e.g., posting $\mathsf{X}(b,i,j)$ or $\mathsf{T}(b,i,ai)$), the honest party must have at hand the revocation keys and signed transactions needed to respond consistently on that ledger and, if necessary, across the others. In this sense, figure 6 is not just a technical depiction of transaction dependencies, but also a visualization of the multi-ledger *watch-and-respond duty* imposed on the parties, a necessary cost to guarantee atomicity and fairness in heterogeneous blockchain environments.

3.2 Our payment channel Protocol

We finally present a formal description of our protocol in Figure 3.1, and prove its correctness and security in Theorem 1 and Theorem 2.

Figure 3.1: Payment Channel Protocol Π_{PCM}

Setup$(\mathsf{B}, \mathsf{F_{P1}}, \mathsf{F_{P2}}, \mathsf{N}, \Delta_{\mathsf{B}})$

1. Bob generates the revocation keys $r^{\mathsf{B}}_{0,1}, r^{\mathsf{B}}_{1,j} \forall_{j\in\{1,\dots,\mathsf{N}\}}$ which he stores in $\mathsf{priv_2}$ and sends to Alice:
   1: the rev-key has $S \leftarrow \mathcal{H}(r^{\mathsf{B}}_{0,1})$. In the setup phase only one possible state exists.
   2: the rev-key hashes $M_{1,j} \leftarrow \mathcal{H}(r^{\mathsf{B}}_{1,j})\ \forall_{j\in\{1,\dots,\mathsf{N}\}}$
   3: the parameters $\mathsf{F_{P2}}, \mathsf{N}, \Delta_{\mathsf{B}}$.
2. Alice generates the revocation key r^{A}_0 which she stores in $\mathsf{priv_1}$ and sends to Bob the following:
   1: the rev-key hash $h_{r^{\mathsf{A}}_0} \leftarrow \mathcal{H}(r^{\mathsf{A}}_0)$.
   2: $\mathsf{F_{P1}}$.
   3: the unsigned transaction $[\mathsf{T}_{(*,0,0)}]$.
   4: the partially signed transactions $[\mathsf{W}_{(*,0,0)}]_{(\mathsf{A},\bullet)}, [\mathsf{X}_{(*,0,0)}]_{(\mathsf{A},\bullet)}, [\mathsf{Y}\circ_{(*,0,0)}]_{(\mathsf{A},\bullet)}$.
3. Bob sends back to Alice the following:
   1: the transaction T with his signature i.e., $[\mathsf{T}_{(*,0,0)}]_{(\bullet,\mathsf{B})}$.
   2: the rev-key hashes $M_{2,j} \leftarrow \mathcal{H}(r^{\mathsf{B}}_{2,j})\ \forall_{j\in\{1,\dots,\mathsf{N}\}}$
4. Alice sends $[\mathsf{F}_*]_{(\mathsf{A},\bullet)}$ to Bob.
5. Bob signs the set of transactions F_*, thus obtaining $[\mathsf{F}_*]_{(\mathsf{A},\mathsf{B})}$ and broadcasts each transaction on its respective blockchain. He may also send $[\mathsf{F}_*]_{(\mathsf{A},\mathsf{B})}$ to Alice and she can broadcast.

Following the definition 4 the private input of Alice (P1) $\mathsf{priv_1}$ consists of the generated revocation key r^{A}_0 and Bob's (P2) private input $\mathsf{priv_2}$ consists of revocation keys $r^{\mathsf{B}}_{0,1}, r^{\mathsf{B}}_{1,j} \forall_{j\in\{1,\dots,\mathsf{N}\}}$. While the state st consists of all the information the parties exchanged in those 5 rounds. The mappings $\mathsf{F_{P1}}, \mathsf{F_{P2}}$ are used to construct the transactions $\mathsf{X}, \mathsf{Y}\circ$ as well as the funding transaction F.

Update$(\mathsf{st}, [(\mathsf{F'_{P1}}, \mathsf{F'_{P2}})]_{\mathsf{N}}, \mathsf{priv_1})$ To propose N new possible states to move the channel from some state(st) $i, i \in \{1,2,3,\dots\}$ to $i+1$:

1. Alice samples the revocation key r^{A}_i (she stores it in $\mathsf{priv_1}$) and generates $[(\mathsf{F'_{P1}}, \mathsf{F'_{P2}})]_{\mathsf{N}}$, which maps the new balances that Alice proposes for both parties. For each proposed state she uses one element of list $[(\mathsf{F'_{P1}}, \mathsf{F'_{P2}})]_{\mathsf{N}}$ to construct transactions $\mathsf{X}, \mathsf{Y}\circ, \mathsf{Y}+$ as described below. Subsequently she sends to Bob the following information ($\mathsf{proposals}$):
   1: the rev-key hash $h_{r^{\mathsf{A}}_i} \leftarrow \mathcal{H}(r^{\mathsf{A}}_i)$.
   2: the partially signed transactions $[\mathsf{W}_{(*,i,j)}]_{(\mathsf{A},\bullet)}, [\mathsf{X}_{(*,i,j)}]_{(\mathsf{A},\bullet)}, [\mathsf{Y}\circ_{(*,i,j)}]_{(\mathsf{A},\bullet)}, [\mathsf{Y}+_{(*,i-1,j)}]_{(\mathsf{A},\bullet)}$ $\forall_{j\in\{1,\dots,K\}}, K \leq \mathsf{N}$.
   3: the rev-key for previous state r^{A}_{i-1}.

[1] This visual notation is adapted from the transaction charts of [Aum+21a; AAM22].

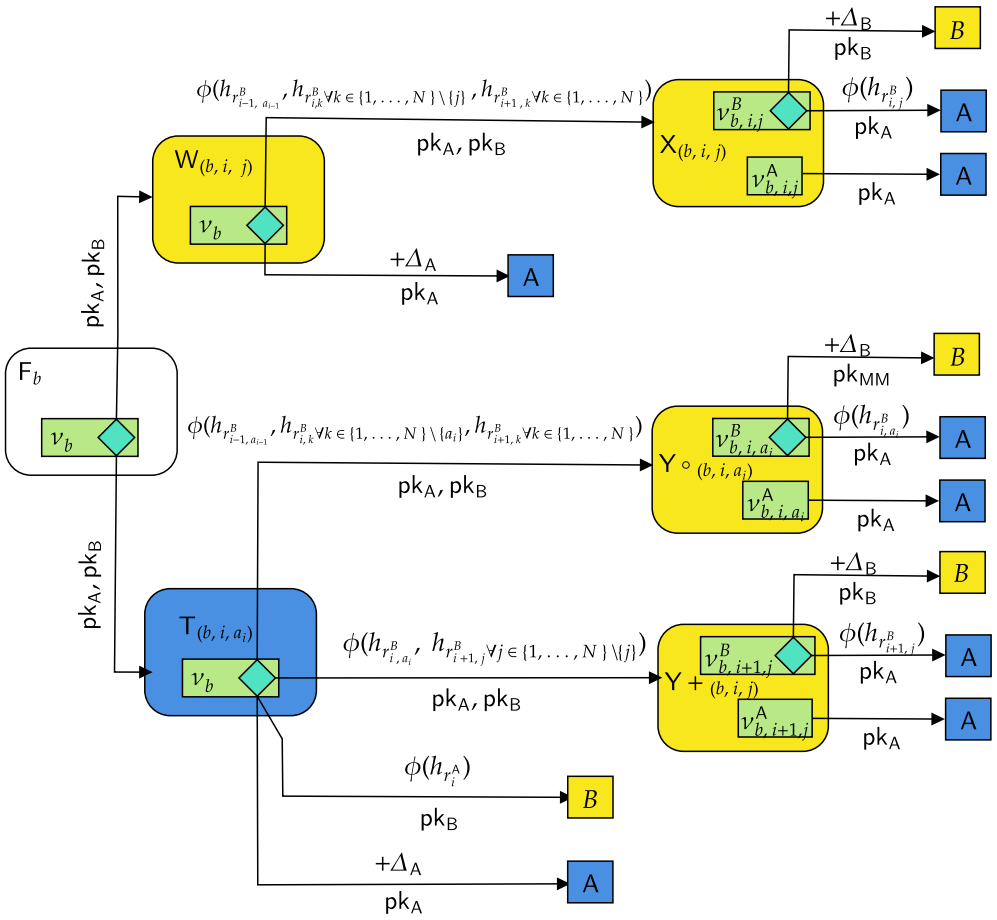

Figure 6: Overview of our design. Rounded rectangles are transactions; blue background e.g., for $\mathsf{T}_{(b,i,a_i)}$ indicates that only Alice should be able to broadcast it, and yellow background e.g., for $\mathsf{X}_{(b,i,j)}$ indicates that only Bob should be able to broadcast it. Transaction $\mathsf{X}_{(b,i,j)}$ has two outputs $\nu^{\mathsf{A}}_{b,i,j}$, and $\nu^{\mathsf{B}}_{b,i,j}$. Output $\nu^{\mathsf{A}}_{b,i,j}$ can be spent by a transaction that verifies w.r.t. $\mathtt{pk}_{\mathsf{A}}$ (a spending transaction needs to verify w.r.t. list of public keys under the arrow). The other output $\nu^{\mathsf{B}}_{b,i,j}$ can be spent in one of the two ways—a diamond indicates an OR relationship between outgoing arrows—(1) by presenting a signature that verifies against $\mathtt{pk}_{\mathsf{B}}$ after Δ_{B} rounds have passed since $\mathsf{X}_{(b,i,j)}$ was posted on-chain; the text above the arrow is the predicate(s) that needs to be satisfied, in this case $+\Delta_{\mathsf{B}}$, and (2) by presenting a signature that verifies against $\mathtt{pk}_{\mathsf{A}}$ and showing preimage of $h_{r^{\mathsf{B}}_{i,j}}$. We use predicate ϕ to require preimage of one ore more hashes. Note that inputs for F_b are not shown, and F_b is not assigned blue or yellow color because it can be broadcast by either party.

UpdateProceed($\mathsf{st}, \mathsf{proposals}, \mathsf{index}, \mathsf{priv}_2$) To update from some state $i, i \in \{1, 2, 3, \dots\}$ to $i+1$, by choosing the $\mathsf{proposals}[index]$ from which he extracts $[\mathsf{T}_{(*,i,index)}]$ and setting $a_i := index$, proceed as follows:

1. Bob generates $r^{\mathsf{B}}_{i+2,j} \forall_{j \in \{1,\dots,\mathsf{N}\}}$ and stores it in priv_2. Subsequently, he sends the new state(st') constructed with hash M_{i,a_i} to Alice. It consists of:
   1: the transaction T with his signature i.e., $[\mathsf{T}_{(*,i,a_i)}]_{(\bullet,\mathsf{B})}$.
   2: the rev-key hash $M_{i+2,j} \leftarrow \mathcal{H}(r^{\mathsf{B}}_{i+2,j})\ \forall_{j \in \{1,\dots,\mathsf{N}\}}$.
   3: the rev-key for previous state $r^{\mathsf{B}}_{i-1,a_{i-1}}$
   4: the rev-keys for current state $r^{\mathsf{B}}_{i,j}\ \forall j \in \{1, \dots, \mathsf{N}\} \setminus \{a_i\}$.

ReqClose(B, st) For some state $i, i \in \{1, 2, 3, \dots\}$, proceed as follows:

1: **if** Alice wants to request the closing of the channel **then**
2: She broadcasts $[\mathsf{T}_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$
3: **end if**
4: **if** Bob wants to request the closing of the channel **then**
5: **if** he has not yet spoken in UpdateProceed for $i-1$ to i **then**
6: He broadcasts $[\mathsf{W}_{(*,i,j)}]_{(\mathsf{A},\mathsf{B})}$, for any trade proposal from Alice on state i
7: **end if**
8: **if** Bob wants to settle at $i-1$ **then**
9: He broadcasts $[\mathsf{W}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$
10: **end if**
11: **end if**

Close($\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B}$) For some state st indexed by i, where $i \in \{1, 2, 3, \dots\}$, in order to respond to ReqClose using the private inputs $\mathsf{priv}_1, \mathsf{priv}_2$—which are used to extract revocation keys not shared with the counterparty previously—and to initiate the broadcast of closing transactions on each $b \in \mathsf{B}$, proceed as follows:

1: Bob parses priv_2 and locally stores $r^{\mathsf{B}}_{k,j} \forall j \in \{1, \dots, \mathsf{N}\} \forall k \in \{i-2, \dots, i+1\}$
2: **if** Bob has not yet spoken in UpdateProceed for $i-1$ to i **then**
3: **if** Alice wants to close the channel **then**
4: She broadcasts $[\mathsf{T}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$
5: **if** Bob wants to settle at i **then**
6: He broadcasts $[\mathsf{Y}+_{(*,i-1,a_i)}]_{(\mathsf{A},\mathsf{B})}$, revealing $r^{\mathsf{B}}_{i-1,a_{i-1}}$, $r^{\mathsf{B}}_{i,j}\ \forall j \in \{1, \dots, \mathsf{N}\} \setminus \{a_i\}$.
7: **end if**
8: **if** Bob wants to settle at $i-1$: **then**
9: He broadcasts $[\mathsf{Y}\circ_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$, revealing $r^{\mathsf{B}}_{i-2,a_{i-2}}$, $r^{\mathsf{B}}_{i-1,j}\ \forall j \in \{1, \dots, \mathsf{N}\} \setminus \{a_{i-1}\}$,$r^{\mathsf{B}}_{i,j}$ $\forall j \in \{1, \dots, \mathsf{N}\}$.
10: **end if**
11: **end if**
12: **end if**
13: **if** Bob wants to close the channel **then**
14: He first broadcasts $[\mathsf{W}_{(*,i,j)}]_{(\mathsf{A},\mathsf{B})}$, and then broadcasts $[\mathsf{X}_{(*,i,j)}]_{(\mathsf{A},\mathsf{B})}$ for any trade proposal from Alice. By doing so, he reveals ($r^{\mathsf{B}}_{i-1,a_{i-1}}$,$r^{\mathsf{B}}_{i,k}\ \forall k \in \{1, \dots, \mathsf{N}\} \setminus \{j\}$,$r^{\mathsf{B}}_{i+1,k}\ \forall k \in \{1, \dots, \mathsf{N}\}$)
15: **end if**
16: **if** Bob wants to settle at $i-1$ **then**
17: He first broadcasts $[\mathsf{W}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$, and then broadcasts $[\mathsf{X}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$. By doing so, he reveals ($r^{\mathsf{B}}_{i-2,a_{i-2}}$,$r^{\mathsf{B}}_{i-1,k}\ \forall k \in \{1, \dots, \mathsf{N}\} \setminus \{a_{i-1}\}$,$r^{\mathsf{B}}_{i,k}\ \forall k \in \{1, \dots, \mathsf{N}\}$)
18: **end if**
19: **if** Bob has spoken in UpdateProceed for $i-1$ to i i.e., the channel state is confirmed to be i **then**
20: **if** Alice wants to close the channel **then**

She can broadcast either $[\mathsf{T}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$ or $[\mathsf{T}_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$.
if Alice broadcasts $[\mathsf{T}_{(*,i-1,a_{i-1})}]_{(\mathsf{A},\mathsf{B})}$ **then**
Bob broadcasts $[\mathsf{Y}+_{(*,i-1,a_i)}]_{(\mathsf{A},\mathsf{B})}$, revealing $(r^{\mathsf{B}}_{i-1,a_{i-1}}, r^{\mathsf{B}}_{i,j}\ \forall j \in \{1,\dots,\mathsf{N}\} \setminus \{a_i\})$.
end if
if Alice broadcasts $[\mathsf{T}_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$ **then**
Bob broadcasts $[\mathsf{Y}\circ_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$, revealing $r^{\mathsf{B}}_{i-1,a_{i-1}}$, $r^{\mathsf{B}}_{i,j}\ \forall j \in \{1,\dots,\mathsf{N}\} \setminus \{a_i\}$,$r^{\mathsf{B}}_{i+1,j}$ $\forall j \in \{1,\dots,\mathsf{N}\}$.
end if
end if
if Bob wants to close the channel **then**
He first broadcasts $[\mathsf{W}_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$, and then broadcasts $[\mathsf{X}_{(*,i,a_i)}]_{(\mathsf{A},\mathsf{B})}$. By doing so, he reveals $(r^{\mathsf{B}}_{i-1,a_{i-1}}$,$r^{\mathsf{B}}_{i,k}\ \forall k \in \{1,\dots,\mathsf{N}\} \setminus \{a_i\}$,$r^{\mathsf{B}}_{i+1,k}\ \forall k \in \{1,\dots,\mathsf{N}\})$
end if
end if

Note that during any of those steps $priv_1$ is not used as Alice does not need to reveal any r^{A}_* on chain.

Theorem 1 (Correctness of Π_{PCM})**.** *Let* B *be a set of blockchains that satisfy Definition 3. Then the protocol* Π_{PCM}*, as defined in Figure 3.2, is a valid instantiation of Definition 4.*

Proof. The protocol Π_{PCM} implements the required interface as specified in Definition 4, namely the algorithms `Setup`, `Update`, `UpdateProceed`, `ReqClose`, and `Close`. Each algorithm consumes and produces inputs and outputs exactly as described. Specifically, party $\mathsf{P1}$ (the proposer) invokes `Update`, while $\mathsf{P2}$ (the approver) invokes `UpdateProceed`. Both parties can invoke `ReqClose`. This is ensured by the protocol design: after `Update`, $\mathsf{P2}$ obtains the set of partially signed transactions W; after `UpdateProceed`, $\mathsf{P1}$ receives the transaction T. Hence, both parties possess the necessary artifacts to initiate channel settlement and to finalize it $(\mathsf{X}, \mathsf{Y}\circ)$.

Upon execution of `ReqClose`, the honest party holds sufficient information to invoke the appropriate closing transaction. In particular, $\mathsf{P2}$ maintains in priv_2 the required revocation keys to broadcast one of the transactions X, Y°, or Y^+, depending on the closure path. $\mathsf{P1}$ can immediately claim his corresponding funds, while $\mathsf{P2}$ is required to wait an additional Δ_B before claiming their balance. The only way $\mathsf{P2}$ could fail to finalize the settlement is if the closing transaction is not included on-chain within Δ_A time slots. However, assuming the underlying blockchains satisfy the liveness property (Definition 3), the probability of such a failure is at most $\mathsf{negl}(\lambda)$. Therefore,

$$\Pr\left[\texttt{Close}(\mathsf{priv}_1, \mathsf{priv}_2, \mathsf{st}, \mathsf{B})\right] \geq 1 - \mathsf{negl}(\lambda)$$

which reflects the probability of the funds described in st to be returned to their rightful owners.

□

Theorem 2 (Security of Π_{PCM})**.** *Under the assumptions that the hash function* $\mathcal{H}$ *behaves as a random oracle and that the digital signature schemes used on each blockchain* $b \in \mathsf{B}$ *(Definition 3) are UF-CMA secure (Definition 2), then* Π_{PCM} *achieves responsive closure, proposer protection, and approver protection.*

Proof. **Responsive Closure.** Let $\mathcal{A}$ be a PPT adversary that statically corrupts either $\mathsf{P1}$ or $\mathsf{P2}$ in the experiment $\mathsf{Exp}^{\mathcal{A}}_{\mathsf{RC},(\mathsf{B},\mathsf{F}_{\mathsf{P1}},\mathsf{F}_{\mathsf{P2}},\Delta_{\mathsf{B}},\mathsf{N})}(\lambda)$ defined in Figure 3. The adversary outputs st_{final} and initiates settlement through `ReqClose`. We consider the two possible cases of corruption separately:

Case 1 (Corrupted $\mathsf{P1}$*).* The adversary can post either $[\mathsf{T}_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ or $[\mathsf{T}_{(*,final-1,a_{final-1})}]_{(\mathsf{A},\mathsf{B})}$. The honest P_2 has Δ_A time to respond with either $[\mathsf{Y}\circ_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ or $[\mathsf{Y}+_{(*,final-1,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ to enforce closure in the latest agreed-upon state st.

Case 2 (Corrupted $\mathsf{P2}$*).* The adversary can post either $[\mathsf{W}_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ if he executed `UpdateProceed` or $[\mathsf{W}_{(*,final,j)}]_{(\mathsf{A},\mathsf{B})} \forall j \in \{1,\dots,\mathsf{N}\}$ if he aborted. Then $\mathsf{P2}$ has Δ_A time to respond with either $[\mathsf{X}_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ or $[\mathsf{X}_{(*,final,j)}]_{(\mathsf{A},\mathsf{B})}$ to enforce closure in the latest agreed-upon state st.

After the broadcast of $\mathsf{Y}\circ$, $\mathsf{Y}+$ or X , $\mathsf{P1}$ can claim their funds immediately, while $\mathsf{P2}$ may need to wait an additional Δ_B before doing so. $\mathcal{A}$ wins the game if the honest party doesn't retrieve their funds. We now analyze the probability that the honest party fails to retrieve their rightful balance. This can only occur in one of the following cases:

1. The adversary attempts to complete a settlement transaction using a revocation key that was not revealed during the protocol execution. Specifically, a malicious $\mathsf{P1}$ can claim the entire channel balance by successfully using the revocation key r^{B}_{i,a_i} for $\mathsf{Y}\circ$ or r^{B}_{i+1,a_j} for $\mathsf{Y}+$, and a malicious $\mathsf{P2}$ can do the same using r^{A}_i. However, revocation keys are sampled uniformly at random from $\{0,1\}^\lambda$ and are revealed only during the finalization of a newer state. Since the settlement occurs at the most recent mutually agreed state, the corresponding revocation keys remain hidden by their respective owners. The hash commitments $H(r)$ are public and H is modeled as a random oracle, the adversary's only option is to query the oracle with the correct preimage. Assuming $\mathcal{A}$ makes at most Q (where Q is polynomial in λ) queries to the random oracle, the probability of guessing the correct preimage is at most $Q/2^\lambda$, which is negligible in the security parameter λ. We denote this probability by Pr_1.
2. The adversary forges a valid signature on a transaction that requires the honest party's authorization in order to settle or claim funds. This would violate the existential unforgeability of the digital signature scheme under chosen message attacks, which we assume holds for all blockchains $b \in \mathsf{B}$. Hence, the probability of such an event is negligible in the security parameter λ. We denote this probability by Pr_2.
3. In case of corrupted $\mathsf{P1}$, the adversary also wins if $\mathsf{P2}$ broadcasted $\mathsf{Y}\circ$ or $\mathsf{Y}+$ but they were not included in the ledger within Δ_A blocks, but this comes with violating the liveness property, and is negligible in the security parameter λ. We denote this probability by Pr_3.

Combining these, the total probability that the adversary wins the responsive closure experiment is bounded by:

$$\Pr\left[\mathsf{Exp}^{\mathcal{A}}_{\mathsf{RC},(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)=1\right] \leq Pr_1 + Pr_2 + Pr_3 = \mathsf{negl}(\lambda).$$

Therefore, Π_{PCM} satisfies responsive closure.

Proposer protection. Let $\mathcal{A}$ be a PPT adversary that statically corrupts the approver $\mathsf{P2}$, while the proposer $\mathsf{P1}$ is honest, in the experiment $\mathsf{Exp}^{\mathcal{A}}_{\mathsf{PP},(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)$. At the end of the game, the adversary outputs a settlement state $\mathsf{st}_{\text{final}}$ and attempts to finalize settlement on-chain using $[\mathsf{W}_{(*,\mathsf{final},a_{\mathsf{final}})}]_{(\mathsf{A},\mathsf{B})}$. There are three possible ways in which $\mathcal{A}$ could win the game:

1. **Forgery of $\mathsf{P1}$'s signature.** The adversary posts a settlement transaction signed by both parties at state $\mathsf{st}_{\text{final}}$, even though $\mathsf{P1}$ never invoked `Update` or signed such a transaction. In this case, the adversary has forged $\mathsf{P1}$'s signature. Note that in this case the adversary can forge a transaction that consumes v_b and doesn't need to follow the specification of W and X. This violates the existential unforgeability of the signature scheme, and occurs with negligible probability. We denote this probability by Pr_1.
2. **Older state posting.** Let $L = [\mathsf{st}_0, \ldots, \mathsf{st}_{\text{final}}, \ldots, \mathsf{st}]$ denote the sequence of protocol-generated states. If $\mathsf{st}_{\text{final}}$ is an older state proposed by $\mathsf{P1}$ and accepted by $\mathsf{P2}$ in the past, but not the most recent one, then $\mathsf{P1}$ holds the revocation key $r^{\mathsf{B}}_{\mathsf{st}_{\text{final}},a_{\mathsf{final}}}$. Upon detecting settlement at this state, $\mathsf{P1}$ can issue a revocation transaction to claim the full channel balance, punishing the adversary. The adversary in this case will win if this dispute transaction does not get included within Δ_B blocks, but this comes with violating the liveness property, and is negligible in the security parameter λ. We denote this probability by Pr_2.
3. **Settle at different proposals.** If the adversary $\mathcal{A}$ aborts during the last execution of `Update` by $\mathsf{P1}$, he retains the ability to settle at any proposal index of their choice by posting a transaction $[\mathsf{W}_{(*,final,j)}]_{(\mathsf{A},\mathsf{B})}$ for some $j \in \{1,\ldots,\mathsf{N}\}$. The adversary selects two indices $j_1, j_2 \in \{1,\ldots,\mathsf{N}\}$ such that $j_1 \neq j_2$ and two distinct blockchains $b_1, b_2 \in \mathsf{B}$ with $b_1 \neq b_2$, and posts the transactions $[\mathsf{W}_{(b_1,final,j_1)}]_{(\mathsf{A},\mathsf{B})}$ and $[\mathsf{W}_{(b_2,final,j_2)}]_{(\mathsf{A},\mathsf{B})}$, attempting to settle in two different proposals across two different blockchains. When $[\mathsf{X}_{(b_1,final,j_1)}]_{(\mathsf{A},\mathsf{B})}$ is posted on-chain, $\mathcal{A}$ has revealed the revocation keys $r^{\mathsf{B}}_{\mathsf{final},k}$ for all $k \in \{1,\ldots,\mathsf{N}\} \setminus \{j_1\}$. Likewise, once $[\mathsf{X}_{(b_2,final,j_2)}]_{(\mathsf{A},\mathsf{B})}$ appears on-chain, $\mathsf{P2}$ has revealed

the keys $r^{\mathsf{B}}_{\mathsf{final},k}$ for all $k \in \{1,\ldots,\mathsf{N}\} \setminus \{j_2\}$. As a result, P1 obtains all revocation keys $r^{\mathsf{B}}_{\mathsf{final},k}$ for $k \in \{1,\ldots,\mathsf{N}\}$. Upon X being included on chain (any chain) , P1 directly claims her balance, she also claims P2's balance by presenting the corresponding revocation key timely, within Δ_B blocks. The adversary succeeds in this attack only if the dispute transactions are not included in the corresponding ledger in time, which would constitute a violation of the liveness property of the blockchains in B—an event that occurs with at most negligible probability in the security parameter λ. We denote this probability by Pr_3.

Combining all cases, the probability that a malicious approver P2 wins the proposer protection sequrity game is bounded by:

$$\Pr\Big[\mathsf{Exp}^{\mathcal{A}}_{\mathsf{PP},(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)=1\Big] \le Pr_1 + Pr_2 + Pr_3 = \mathsf{negl}(\lambda).$$

Hence, Π_{PCM} satisfies proposer protection.

Approver protection. Let $\mathcal{A}$ be a PPT adversary that statically corrupts the proposer P1, while the approver P2 is honest, in the experiment $\mathsf{Exp}^{\mathcal{A}}_{\mathsf{AP},(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)$. At the end of the game, the adversary outputs a state st and attempts to finalize settlement on-chain at that state $\mathsf{st}_{\mathsf{final}}$ by posting $[\mathsf{T}_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$.

There are three possible cases for $\mathcal{A}$ to win the game:

1. **Forgery of P2's signature.** If the adversary is able to produce a valid settlement transaction $[\mathsf{T}_{(*,final,a_{final})}]_{(\mathsf{A},\mathsf{B})}$ that includes a signature from P2, but P2 never participated in `UpdateProceed` for that state, then $\mathcal{A}$ has forged a signature. In that case, P2 will not have transactions Y∘ or Y+ for st_{final}, and thus after Δ_A P1 will claim the entire balance of the channel. This path though, contradicts the existential unforgeability of the underlying digital signature scheme, and occurs with probability at most $\mathsf{negl}(\lambda)$. We denote this probability by Pr_1.
2. **Older state posting.** Let the list $L = [st_0,\ldots,\mathsf{st}_{\mathrm{final}},\ldots,\mathsf{st}]$ represent the sequence of states that were generated by the protocol. Since P2 behaves honestly, he only proceeds with `UpdateProceed` when he has recieved the revocation key of P1 for the previous state. In particular, if $\mathsf{st}_{\mathrm{final}}$ is not the last confirmed state via `UpdateProceed`, then P2 holds the revocation key $r^{\mathsf{A}}_{\mathsf{st}_{\mathrm{final}}}$. Therefore, P2 can broadcast a revocation transaction containing $r^{\mathsf{A}}_{\mathsf{final}}$, thereby claiming the entire balance of the channel as a penalty to the misbehaving proposer. $\mathcal{A}$ wins if this dispute transaction, does not get included within Δ_A blocks, which contradicts with the liveness property and occurs with probability at most $\mathsf{negl}(\lambda)$. Since $\mathcal{A}$ cannot settle in older states , this means that he cannot settle in different states accross different blockchains, as the revocation keys are the same and P1 has only one revocation key per state. We denote this probability by Pr_2.
3. **Forgery / Revocation key guessing.** Let the list $L = [\mathsf{st}_0,\ldots,\mathsf{st}_{\mathrm{final}}]$ represent the sequence of states that were generated by the protocol. Since P2 behaves honestly, they will respond to any valid settlement attempt by posting either $[\mathsf{Y}\circ_{(*,\mathsf{final},a_{\mathsf{final}})}]_{(\mathsf{A},\mathsf{B})}$ or, if $\mathsf{st}_{\mathrm{final}}$ is the penultimate state, the completion transaction $[\mathsf{Y}+_{(*,\mathsf{final}-1,a_{\mathsf{final}})}]_{(\mathsf{A},\mathsf{B})}$, finalizing settlement at the latest agreed state.
   The adversary can win in two subcases:
   (a) **Forgery.** The adversary successfully forges a claim transaction. This breaks the existential unforgeability of the digital signature scheme, which occurs with negligible probability $\mathsf{negl}(\lambda)$. We denote this probability by Pr_{3a}.
   (b) **Revocation key guessing.** The adversary attempts to guess the revocation key that P2 maintains private. Since the hash commitments $H(r)$ are public and H is modeled as a random oracle, the adversary's only option is to query the oracle on the correct preimage. Assuming the adversary makes at most Q (where Q is polynomial in λ) queries, the probability of success is at most $Q/2^\lambda$, which is negligible in the security parameter λ. We denote this probability by Pr_{3b}.

Combining all cases, the probability that a malicious proposer P1 wins the approver protection security game is bounded by:

$$\Pr\Big[\mathsf{Exp}^{\mathcal{A}}_{\mathsf{AP},(\mathsf{B},\mathsf{F_{P1}},\mathsf{F_{P2}},\Delta_\mathsf{B},\mathsf{N})}(\lambda)=1\Big] \le Pr_1 + Pr_2 + Pr_{3a} + Pr_{3b} = \mathsf{negl}(\lambda).$$

Hence, Π_{PCM} satisfies approver protection. □

4 Implementation Details

The blockchains chosen for the development of Π_{PCM} are Ethereum, Bitcoin, and Algorand. The programming languages used for the diverse needs of Π_{PCM} are Node.js v16.20.2[2], Solidity v0.8.17[3], Bitcoin Script, PyTeal v0.24.1[4] and Bash. The key components are:

1. **Server:** The server encapsulates the role of approver (Bob). It listens to proposer's (Alice) messages for channel setup and updates, and processes them according to the protocol specification. For its storage needs, we use a PostgreSQL database. We use web-sockets for communication due to their low overhead.
2. **Client:** The client encapsulates the role of a proposer (Alice). It communicates with the server through web-sockets. Following the protocol specification Π_{PCM}, it enables Alice to (1) register their payment channels with Bob, (2) exchange trade messages and, accordingly, update the payment channels, (3) settle the payment channels, etc. The client uses a SQLite instance for storage purposes.
3. **Monitor:** Both the proposer and the apporver run a Monitor. A monitor is an *observer* (or publisher) that continuously monitors the blockchain on behalf of its owner. The monitor subscribes to events of his interest, e.g., a certain transaction entering the mempool. Whenever one of the subscribed events is detected, it notifies its owner. The fact that every blockchain is different means that event detection needs unique implementation for each case. For example, in the case of Ethereum and Algorand, we detect events by tracking changes in the contract's state, but in the case of Bitcoin, we monitor the transaction IDs that enter the mempool. Note that, in contrast to the generic monitors e.g., [ALW20; Mir+21; Ava+18; KNW19; McC+19; Mir+22b], this service is private and tailored to our own system.
4. **Workers:** Worker is the *subscriber* to the Monitor. Similar to the Monitor, it runs within each client and server. Upon notification from the Monitor e.g., upon observing the transaction $[\mathsf{Y}\circ_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$ reaching the blockchain, the worker would take an appropriate action. In this example, the worker would post appropriate revocation key $r^{\mathsf{B}}_{i,j}$ to claim cheating if the posted $[\mathsf{Y}\circ_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$ is not the last mutually agreed state.

We use the following software to realize private blockchains in our testing:

1. **Bitcoin-Core v22.0[5]:** This is the reference implementation of Bitcoin protocol. We run it in `regtest` mode, and with ZeroMQ (`zmq`) enabled. ZeroMQ's flexible and fine-grained publish-subscribe model allows components like our Monitor to receive only relevant messages, keeping them lightweight and efficient.
2. **Ganache v7.7.3[6]:** Ganache is a developer-friendly implementation of Ethereum that provides a REST API for interacting with a local blockchain, allowing developers to deploy contracts, inspect state, and test transactions in a controlled environment. We use exactly this API, via web-sockets, in our Monitor to observe events on the Ethereum blockchain.
3. **Algokit v2.6.2[7]:** We use Algokit to deploy a private Algorand blockchain, precisely following the guidelines in the official documentation. Among our selected blockchains, observing events on Algorand proved to be the most challenging, as there is no built-in or black-box way to monitor state changes. To address this, we implemented a custom, fine-grained subsystem that analyzes each block in detail. When events of interest are detected, the system triggers appropriate notifications to subscribed components. We note that further development of this subsystem, beyond the scope of this work, could be of independent interest.

For UTXO-based blockchains such as Bitcoin, we realize the payment channel by directly constructing transactions according to the protocol Π_{PCM}, and we implement them using P2WSH (Pay to Witness Script Hash), a SegWit feature that enables complex spending conditions to be encoded efficiently. P2WSH offers benefits such as reduced transaction size that lead to lower fees. However, it also imposes a limitation:

[2] https://nodejs.org/en/blog/release/v16.20.2
[3] https://docs.soliditylang.org/en/v0.8.17/
[4] https://pypi.org/project/pyteal/0.24.1/
[5] https://bitcoincore.org/bin/bitcoin-core-22.0
[6] https://www.npmjs.com/package/ganache/v/7.7.3
[7] https://pypi.org/project/algokit/2.6.2

the witness script may contain at most 201 non-push opcodes, which constrains the expressiveness of the script. This limitation prevents us from scaling the number of total proposals (N) arbitrarily—specifically, we measure up to 16 proposals, as testing 32 introduces script complexity that exceeds the opcode limit. For account-based blockchains like Ethereum and Algorand, we implement the payment channel using smart contracts. These contracts are designed with interfaces that closely emulate the transaction semantics of the UTXO model. For example, the smart contract accepts a message W in place of the transaction W, and similarly for the other components of the protocol. While Solidity provides sufficient expressiveness to define complex data structures and store them directly within a contract's state, PyTeal operates under more restrictive constraints. In particular, Algorand smart contracts are limited to 64 global and 16 local key-value pairs, with each key and value capped at 128 bytes. However, our protocol requires a dynamic and scalable storage model to accommodate an increasing number of proposals, as more hashes must be stored and validated on-chain. To address this, we leverage boxes—a feature in Algorand that enables storage of up to 32KB per key. The use of boxes, however, incurs a Minimum Balance Requirement (MBR): the contract must hold additional funds proportional to the total allocated storage. These funds are locked for the duration of the box's existence but are refunded to the contract owner upon its deletion. Both our Algorand and Ethereum implemetation can scale as N increases without the need for modifications.

4.1 Experiment Setup

We conduct our experiments on a MacBook Pro equipped with an Apple M1 Max processor and 64GB of RAM. Our deployment includes three blockchains Bitcoin, Ethereum, and Algorand as well as the Monitor, the server (Bob) and his corresponding worker, and the client (Alice) with her respective worker. In this setup, Alice and Bob establish payment channels on all three chains. To the best of our knowledge, this is the first system to enable cross-chain atomic swaps in a payment channel setting that is fully compatible with Bitcoin. As such, our construction is not directly comparable to prior works, many of which are either limited to single-chain deployments, support only a single proposal per update, or rely on expressive smart contract platforms incompatible with Bitcoin. To understand the cost implications of our construction, we begin by presenting a set of tables, focusing on the most resource-intensive components—namely, the *opening* and *closing* and *disputing* transactions. The remaining components of the protocol are off-chain and scale with available computational resources. To obtain accurate cost estimates, we developed a custom testing suite that systematically exercises all possible settlement paths across each supported blockchain and logs the corresponding on-chain costs per operation. We use a security parameter of $\lambda := 160$ bits to generate revocation keys.

4.2 Results and Analysis

We report the on-chain costs associated with each transaction defined in Π_{PCM}, averaged over 10 executions of the same operation. Our measurements are conducted for $\mathsf{N} \in \{1, 2, 4, 8, 16\}$. We adopt the notation used in the protocol specification for the main transaction types—namely, T, W, X, $\mathsf{Y}\circ$, and $\mathsf{Y}+$. To distinguish between parties and their roles in the dispute-resolution process, we annotate transactions as follows: Tx_A denotes a dispute transaction initiated by Alice on transaction Tx, and Tx_B denotes the corresponding action by Bob. We further use Tx_{CA} and Tx_{CB} to denote claim transactions initiated by Alice and Bob, respectively. Moreover, while the funding step F in Bitcoin adheres directly to the protocol specification, the implementation on Ethereum and Algorand requires each party to independently initiate a transaction to deposit their respective funds. We denote these transactions as F_1 for Alice and F_2 for Bob. The measured gas usage and estimated fees for these transactions on Bitcoin and Ethereum are summarized in Table 5 and Table 6 (of the Appendix B), respectively. We additionally present Table 3 and Table 4, which outlines the total fees (in USD) each party must pay, accounting for honest execution and the the most expensive malicious paths.

Our results illustrate that Π_{PCM} achieves cross-chain atomicity, protocol simplicity, and low operational costs—offering a practical and secure foundation for real-world interoperability. These findings demonstrate the feasibility of deploying Bitcoin-compatible payment channels that scale across heterogeneous ecosystems

N	W,X,X_{CB}		W,X,X_A		T,Y$\circ$,Y$\circ_{CB}$		T,Y$\circ$,Y$\circ_A$		T,Y+,Y+$_{CB}$		T,Y+,Y+$_A$	
	A	**B**	**A**	**B**	**A**	**B**	**A**	**B**	**A**	**B**	**A**	**B**
1	0.51	2.03	1.05	1.51	1.13	1.57	1.67	1.05	1.13	1.55	1.67	1.03
2	0.51	2.08	1.06	1.56	1.13	1.61	1.68	1.09	1.13	1.57	1.68	1.05
4	0.51	2.26	1.05	1.74	1.13	1.84	1.67	1.32	1.13	1.75	1.67	1.23
8	0.51	2.63	1.06	2.11	1.13	2.30	1.68	1.78	1.13	2.13	1.68	1.61
16	0.51	3.37	1.06	2.84	1.13	3.24	1.68	2.71	1.13	2.88	1.68	2.35

Table 3: Π_{PCM} settlement paths costs($) on Bitcoin for Alice (column A) and Bob (column B). At the time of writing, the fee rate was 4.0 sat/vB (source: https://btc.network/estimate on 2025-05-16) for inclusion withing 6 block, and the BTC/USD exchange rate was $103,747.15 (source: CoinMarketCap on 2025-05-16). The cost of F is always assigned to Alice and is a flat 0.51$. Moreover Alice balance becomes spendable imidiately with no need for her to post a claim transaction

N	W,X,X_{CB}		W,X,X_A		T,Y$\circ$,Y$\circ_{CB}$		T,Y$\circ$,Y$\circ_A$		T,Y+,Y+$_{CB}$		T,Y+,Y+$_A$	
	A	**B**	**A**	**B**	**A**	**B**	**A**	**B**	**A**	**B**	**A**	**B**
1	0.65	1.68	1.05	1.28	1.53	0.92	1.94	0.51	1.54	1.02	1.95	0.61
2	0.65	2.04	1.09	1.60	1.77	1.02	2.22	0.57	1.78	1.09	2.23	0.64
4	0.65	2.64	1.17	2.13	2.21	1.20	2.74	0.67	2.22	1.22	2.74	0.70
8	0.65	3.81	1.31	3.15	3.03	1.54	3.70	0.87	3.04	1.48	3.71	0.81
16	0.65	6.09	1.60	5.15	4.65	2.22	5.61	1.26	4.66	1.97	5.62	1.01

Table 4: Π_{PCM} settlement paths costs($) on Ethereum for Alice (column A) and Bob (column B). At the time of writing, the ETH/USD exchange rate was $2,588.56 (source: CoinMarketCap on 2025-05-16), and the gas price was 1.72 GWEI. Alice always pays the cost of F1 and Tx_{CA}, while Bob always pay F2, these transactions are omitted from the path visualization to maintain uniformity with Bitcoin's transaction flow, but their associated costs are still included in the overall fee calculations.

without relying on advanced smart contract platforms. We observe that, under moderate network conditions, transaction costs remain low. Furthermore, we note that careful configuration of the protocol parameter Δ_B can further reduce costs, as it allows transactions to be included at a later time—potentially avoiding periods of network congestion—while still maintaining the security of the locked assets.

Our tables also reveal that doubling the number of supported proposals N results in a sublinear increase in transaction costs across both Bitcoin and Ethereum. Although our current implementation adheres to Bitcoin's P2WSH constraint of 201 non-push opcodes, we emphasize that the design leaves substantial headroom before cost or complexity becomes prohibitive. If alternative scripting methods or upcoming upgrades allow circumventing this limitation, our construction could scale even further.

For Algorand, we omit transaction fees due to its flat-fee model and consistently low costs—typically less than 0.01$ at the time of writing. One might argue that the Minimum Balance Requirement (MBR) should be reported instead; however, since this amount is refundable upon box deletion, we do not include it in the cost analysis. For completeness, we include the MBR formula: when a box with name n and size s is created, the MBR is increased by $2500 + 400 \cdot (\texttt{len}(n) + s)$ microAlgos. Upon box destruction, the same amount is refunded. In our protocol, one box is needed and it stores $2 \cdot \frac{\lambda}{8} \cdot \mathsf{N}$ bytes of data where N is the total proposals and λ is the security parameter for sampling the revocation keys.

Acknowledgments Ioannis Tzannetos was funded by AnalytiXIN.

References

[AAM22] Lukas Aumayr, Kasra Abbaszadeh, and Matteo Maffei. "Thora: Atomic and Privacy-Preserving Multi-Channel Updates". In: *ACM CCS 2022*. Ed. by Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi. ACM Press, Nov. 2022, pp. 165–178. DOI: 10.1145/3548606.3560556.

[Aum+24] Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and Subhra Mazumdar. "Securing Lightning Channels against Rational Miners". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 393–407.

[Aum+25] Lukas Aumayr, Zeta Avarikioti, Iosif Salem, Stefan Schmid, and Michelle Yeo. "X-Transfer: Enabling and optimizing cross-PCN transactions". In: *Cryptology ePrint Archive* (2025).

[Aum+21a] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. "Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures". In: *ASIACRYPT 2021, Part II*. Ed. by Mehdi Tibouchi and Huaxiong Wang. Vol. 13091. LNCS. Springer, Heidelberg, Dec. 2021, pp. 635–664. DOI: 10.1007/978-3-030-92075-3_22.

[Aum+21b] Lukas Aumayr, Matteo Maffei, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Siavash Riahi, Kristina Hostáková, and Pedro Moreno-Sanchez. "Bitcoin-Compatible Virtual Channels". In: *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 901–918. DOI: 10.1109/SP40001.2021.00097.

[Aum+21c] Lukas Aumayr, Pedro Moreno-Sanchez, Aniket Kate, and Matteo Maffei. "Blitz: Secure Multi-Hop Payments Without Two-Phase Commits". In: *USENIX Security 2021*. Ed. by Michael Bailey and Rachel Greenstadt. USENIX Association, Aug. 2021, pp. 4043–4060.

[Aum+22] Lukas Aumayr, Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, Pedro Moreno-Sanchez, and Matteo Maffei. "Sleepy Channels: Bi-directional Payment Channels without Watchtowers". In: *ACM CCS 2022*. Ed. by Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi. ACM Press, Nov. 2022, pp. 179–192. DOI: 10.1145/3548606.3559370.

[Ava+18] Georgia Avarikioti, Felix Laufenberg, Jakub Sliwinski, Yuyi Wang, and Roger Wattenhofer. "Towards secure and efficient payment channels". In: *arXiv preprint arXiv:1811.12740* (2018).

[Ava+21] Zeta Avarikioti, Eleftherios Kokoris-Kogias, Roger Wattenhofer, and Dionysis Zindros. "Brick: Asynchronous Incentive-Compatible Payment Channels". In: *FC 2021, Part II*. Ed. by Nikita Borisov and Claudia Díaz. Vol. 12675. LNCS. Springer, Heidelberg, Mar. 2021, pp. 209–230. DOI: 10.1007/978-3-662-64331-0_11.

[ALW20] Zeta Avarikioti, Orfeas Stefanos Thyfronitis Litos, and Roger Wattenhofer. "Cerberus Channels: Incentivizing Watchtowers for Bitcoin". In: *FC 2020*. Ed. by Joseph Bonneau and Nadia Heninger. Vol. 12059. LNCS. Springer, Heidelberg, Feb. 2020, pp. 346–366. DOI: 10.1007/978-3-030-51280-4_19.

[Ava+23] Zeta Avarikioti, Tomasz Lizurej, Tomasz Michalak, and Michelle Yeo. "Lightning creation games". In: *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. IEEE. 2023, pp. 1–11.

[Ava+22] Zeta Avarikioti, Krzysztof Pietrzak, Iosif Salem, Stefan Schmid, Samarth Tiwari, and Michelle Yeo. "Hide & Seek: Privacy-Preserving Rebalancing on Payment Channel Networks". In: *FC 2022*. Ed. by Ittay Eyal and Juan A. Garay. Vol. 13411. LNCS. Springer, Heidelberg, May 2022, pp. 358–373. DOI: 10.1007/978-3-031-18283-9_17.

[AST24] Zeta Avarikioti, Stefan Schmid, and Samarth Tiwari. "Brief Announcement: Musketeer - Incentive-Compatible Rebalancing for Payment Channel Networks". In: *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*. PODC '24. Nantes, France: Association for Computing Machinery, 2024, 306–309. ISBN: 9798400706684. DOI: 10.1145/3662158.3662809. URL: https://doi.org/10.1145/3662158.3662809.

[BDM16] Waclaw Banasik, Stefan Dziembowski, and Daniel Malinowski. "Efficient Zero-Knowledge Contingent Payments in Cryptocurrencies Without Scripts". In: *ESORICS 2016, Part II*. Ed. by Ioannis G. Askoxylakis, Sotiris Ioannidis, Sokratis K. Katsikas, and Catherine A. Meadows. Vol. 9879. LNCS. Springer, Heidelberg, Sept. 2016, pp. 261–280. DOI: 10.1007/978-3-319-45741-3_14.

[Bit25] Bitcoin Wiki contributors. *Script*. https://en.bitcoin.it/wiki/Script. Accessed: 2025-04-30. 2025.

[Cam+17] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. "Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services". In: *ACM CCS 2017*. Ed. by Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM Press, 2017, pp. 229–243. DOI: 10.1145/3133956.3134060.

[Cha24] Chainlink Labs. *Chainlink Cross-Chain Interoperability Protocol (CCIP)*. https://docs.chain.link/ccip. Accessed: 2025-09-03. 2024.

[Cha+21a] Manuel M. T. Chakravarty, Sandro Coretti, Matthias Fitzi, Peter Gazi, Philipp Kant, Aggelos Kiayias, and Alexander Russell. "Fast Isomorphic State Channels". In: *FC 2021, Part II*. Ed. by Nikita Borisov and Claudia Díaz. Vol. 12675. LNCS. Springer, Heidelberg, Mar. 2021, pp. 339–358. DOI: 10.1007/978-3-662-64331-0_18.

[Cha+21b] Manuel MT Chakravarty, Sandro Coretti, Matthias Fitzi, Peter Gaži, Philipp Kant, Aggelos Kiayias, and Alexander Russell. "Fast isomorphic state channels". In: *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II 25*. Springer. 2021, pp. 339–358.

[Cia+22] Michele Ciampi, Muhammad Ishaq, Malik Magdon-Ismail, Rafail Ostrovsky, and Vassilis Zikas. "FairMM: A Fast and Frontrunning-Resistant Crypto Market-Maker". In: *Cyber Security, Cryptology, and Machine Learning*. Springer International Publishing, 2022, pp. 428–446.

[Cia+20] Michele Ciampi, Nikos Karayannidis, Aggelos Kiayias, and Dionysis Zindros. "Updatable Blockchains". In: *Proceedings of the 25th European Symposium on Research in Computer Security (ESORICS 2020), Part II*. Springer. 2020, pp. 590–609.

[Cur20] Curve. *Curve*. 2020. (Visited on 03/10/2021).

[Das05] Sanmay Das. "A learning market-maker in the Glosten–Milgrom model". In: *Quantitative Finance* 5.2 (2005), pp. 169–180.

[DMI08] Sanmay Das and Malik Magdon-Ismail. "Adapting to a market shock: Optimal sequential market-making". In: *Advances in Neural Information Processing Systems* 21 (2008).

[DRO18] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. "eltoo: A simple layer2 protocol for bitcoin". In: *White paper: https://blockstream. com/eltoo. pdf* (2018).

[DW15] Christian Decker and Roger Wattenhofer. "A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels". In: *Stabilization, Safety, and Security of Distributed Systems*. Ed. by Andrzej Pelc and Alexander A. Schwarzmann. Springer International Publishing, 2015, pp. 3–18. DOI: 10.1007/978-3-319-21741-3_1.

[DH20] Apoorvaa Deshpande and Maurice Herlihy. "Privacy-Preserving Cross-Chain Atomic Swaps". In: *FC 2020 Workshops*. Ed. by Matthew Bernhard, Andrea Bracciali, L. Jean Camp, Shin'ichiro Matsuo, Alana Maurushat, Peter B. Rønne, and Massimiliano Sala. Vol. 12063. LNCS. Springer, Heidelberg, Feb. 2020, pp. 540–549. DOI: 10.1007/978-3-030-54455-3_38.

[DEF18] Stefan Dziembowski, Lisa Eckey, and Sebastian Faust. "FairSwap: How To Fairly Exchange Digital Goods". In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 967–984. DOI: 10.1145/3243734.3243857.

[Dzi+19a] Stefan Dziembowski, Lisa Eckey, Sebastian Faust, Julia Hesse, and Kristina Hostáková. "Multi-party Virtual State Channels". In: *EUROCRYPT 2019, Part I*. Ed. by Yuval Ishai and Vincent Rijmen. Vol. 11476. LNCS. Springer, Heidelberg, May 2019, pp. 625–656. DOI: 10.1007/978-3-030-17653-2_21.

[Dzi+19b] Stefan Dziembowski, Lisa Eckey, Sebastian Faust, and Daniel Malinowski. "Perun: Virtual Payment Hubs over Cryptocurrencies". In: *2019 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2019, pp. 106–123. DOI: 10.1109/SP.2019.00020.

[DFH18] Stefan Dziembowski, Sebastian Faust, and Kristina Hostáková. "General State Channel Networks". In: *ACM CCS 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM Press, Oct. 2018, pp. 949–966. DOI: 10.1145/3243734.3243856.

[EFS20] Lisa Eckey, Sebastian Faust, and Benjamin Schlosser. "OptiSwap: Fast Optimistic Fair Exchange". In: *ASIACCS 20*. Ed. by Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese. ACM Press, Oct. 2020, pp. 543–557. DOI: 10.1145/3320269.3384749.

[Esp23] Espresso Systems. *Espresso Sequencer*. https://www.espressosys.com/. Accessed: 2025-09-03. 2023.

[Fuc19] Georg Fuchsbauer. "WI Is Not Enough: Zero-Knowledge Contingent (Service) Payments Revisited". In: *ACM CCS 2019*. Ed. by Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz. ACM Press, Nov. 2019, pp. 49–62. DOI: 10.1145/3319535.3354234.

[Ge+23] Zhonghui Ge, Jiayuan Gu, Chenke Wang, Yu Long, Xian Xu, and Dawu Gu. "Accio: Variable-Amount, Optimized-Unlinkable and NIZK-Free Off-Chain Payments via Hubs". In: *ACM CCS 2022*. Ed. by Weizhi Meng, Christian Jensen, Cas Cremers, and Engin Kirda. ACM Press, Nov. 2023, 1541–1555. DOI: 10.1145/3576915.3616577.

[Gla+22] Noemi Glaeser, Matteo Maffei, Giulio Malavolta, Pedro Moreno-Sanchez, Erkan Tairi, and Sri Aravinda Krishnan Thyagarajan. "Foundations of Coin Mixing Services". In: *ACM CCS 2022*. Ed. by Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi. ACM Press, Nov. 2022, pp. 1259–1273. DOI: 10.1145/3548606.3560637.

[GM17] Matthew Green and Ian Miers. "Bolt: Anonymous Payment Channels for Decentralized Currencies". In: *ACM CCS 2017*. Ed. by Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM Press, 2017, pp. 473–489. DOI: 10.1145/3133956.3134093.

[Han+24] Lucjan Hanzlik, Julian Loss Sri, AravindaKrishnan Thyagarajan, and Benedikt Wagner. "Sweep-uc: Swapping coins privately". In: *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2024, pp. 3822–3839.

[Hei+17] Ethan Heilman, Leen Alshenibr, Foteini Baldimtsi, Alessandra Scafuro, and Sharon Goldberg. "TumbleBit: An Untrusted Bitcoin-Compatible Anonymous Payment Hub". In: *NDSS 2017*. The Internet Society, 2017.

[Her18] Maurice Herlihy. "Atomic Cross-Chain Swaps". In: *37th ACM PODC*. Ed. by Calvin Newport and Idit Keidar. ACM, July 2018, pp. 245–254. DOI: 10.1145/3212734.3212736.

[KNW19] Majid Khabbazian, Tejaswi Nadahalli, and Roger Wattenhofer. "Outpost: A Responsive Lightweight Watchtower". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*. Association for Computing Machinery, 2019, 31–40. DOI: 10.1145/3318041.3355464.

[Kur+] Ahmet Kurt, Kemal Akkaya, Sabri Yilmaz, Suat Mercan, Omer Shlomovits, and Enes Erdin. *LNGate^2: Secure Bidirectional IoT Micro-payments using Bitcoin's Lightning Network and Threshold Cryptography*. URL: http://arxiv.org/abs/2206.02248.

[Li+25] Ming Li, Yuxian Li, Jian Weng, Yingjiu Li, Jiasi Weng, Junzuo Lai, and Robert H. Deng. "IvyAPC: Auditable Generalized Payment Channels". In: *Proceedings of the 29th International Conference on Financial Cryptography and Data Security (FC 2025)*. Miyakojima, Japan, 2025.

[Li+24] Yuxian Li, Jian Weng, Junzuo Lai, Yingjiu Li, Jianfei Sun, Jiahe Wu, Ming Li, Pengfei Wu, and Robert H Deng. "AuditPCH: Auditable Payment Channel Hub With Privacy Protection". In: *IEEE Transactions on Information Forensics and Security* (2024).

[Lia+22] Jinghui Liao, Fengwei Zhang, Wenhai Sun, and Weisong Shi. "Speedster: An Efficient Multi-party State Channel via Enclaves". In: *ASIACCS 22*. Ed. by Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako. ACM Press, 2022, pp. 637–651. DOI: 10.1145/3488932.3523259.

[Lin+] Joshua Lind, Ittay Eyal, Peter Pietzuch, and Emin Gün Sirer. *Teechan: Payment Channels Using Trusted Execution Environments*. DOI: 10.48550/arXiv.1612.07766. URL: http://arxiv.org/abs/1612.07766.

[McC+19] Patrick McCorry, Surya Bakshi, Iddo Bentov, Sarah Meiklejohn, and Andrew Miller. "Pisa: Arbitration Outsourcing for State Channels". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*. Association for Computing Machinery, 2019, 16–30. DOI: 10.1145/3318041.3355461.

[Mil+19] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. "Sprites and State Channels: Payment Networks that Go Faster Than Lightning". In: *FC 2019*. Ed. by Ian Goldberg and Tyler Moore. Vol. 11598. LNCS. Springer, Heidelberg, Feb. 2019, pp. 508–526. DOI: 10.1007/978-3-030-32101-7_30.

[Mir+22a] Arash Mirzaei, Amin Sakzad, Jiangshan Yu, and Ron Steinfeld. *Daric: A Storage Efficient Payment Channel With Penalization Mechanism*. 2022. URL: https://eprint.iacr.org/2022/1295.

[Mir+21] Arash Mirzaei, Amin Sakzad, Jiangshan Yu, and Ron Steinfeld. "FPPW: A Fair and Privacy Preserving Watchtower for Bitcoin". In: *FC 2021, Part II*. Ed. by Nikita Borisov and Claudia Díaz. Vol. 12675. LNCS. Springer, Heidelberg, Mar. 2021, pp. 151–169. DOI: 10.1007/978-3-662-64331-0_8.

[Mir+22b] Arash Mirzaei, Amin Sakzad, Jiangshan Yu, and Ron Steinfeld. *Garrison: A Novel Watchtower Scheme for Bitcoin*. Cryptology ePrint Archive, Paper 2022/1300. 2022. URL: https://eprint.iacr.org/2022/1300.

[Mor+20] Pedro Moreno-Sanchez, Arthur Blue, Duc Viet Le, Sarang Noether, Brandon Goodell, and Aniket Kate. "DLSAG: Non-interactive Refund Transactions for Interoperable Payment Channels in Monero". In: *FC 2020*. Ed. by Joseph Bonneau and Nadia Heninger. Vol. 12059. LNCS. Springer, Heidelberg, Feb. 2020, pp. 325–345. DOI: 10.1007/978-3-030-51280-4_18.

[MS+25] Pedro Moreno-Sanchez, Mohsen Minaei, Srinivasan Raghuraman, Panagiotis Chatzigiannis, and Duc V Le. "AUPCH: Auditable Unlinkable Payment Channel Hubs". In: *Cryptology ePrint Archive* (2025).

[Net18] Raiden Network. *What is Raiden Network?* 2018. URL: https://raiden.network/101.html.

[Pol24] Polygon Labs. *Polygon AggLayer Overview*. https://docs.agglayer.dev/. Accessed: 2025-09-03. 2024.

[PD16] Joseph Poon and Thaddeus Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. 2016. URL: https://lightning.network/lightning-network-paper.pdf.

[Qin+23] Xianrui Qin, Shimin Pan, Arash Mirzaei, Zhimei Sui, Oguzhan Ersoy, Amin Sakzad, Muhammed F. Esgin, Joseph K. Liu, Jiangshan Yu, and Tsz Hon Yuen. "BlindHub: Bitcoin-Compatible Privacy-Preserving Payment Channel Hubs Supporting Variable Amounts". In: *2023 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2023, pp. 2462–2480. DOI: 10.1109/SP46215.2023.10179427.

[Spi14] Jeremy Spilman. *Bitcoin Contracts*. 2014. URL: https://en.bitcoin.it/wiki/Contracts.

[Sui+22] Zhimei Sui, Joseph K. Liu, Jiangshan Yu, Man Ho Au, and Jia Liu. "AuxChannel: Enabling Efficient Bi-Directional Channel for Scriptless Blockchains". In: *ASIACCS 22*. Ed. by Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako. ACM Press, 2022, pp. 138–152. DOI: 10.1145/3488932.3524126.

[TMSM21] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. "A 2 l: Anonymous atomic locks for scalability in payment channel hubs". In: *2021 IEEE symposium on security and privacy (SP)*. IEEE. 2021, pp. 1834–1851.

[TMSS23] Erkan Tairi, Pedro Moreno-Sanchez, and Clara Schneidewind. "LedgerLocks: A Security Framework for Blockchain Protocols Based on Adaptor Signatures". In: *ACM CCS 2022*. Ed. by Weizhi Meng, Christian Jensen, Cas Cremers, and Engin Kirda. ACM Press, Nov. 2023, 859–873. DOI: 10.1145/3576915.3623149.

[Tha24] Thales Group. *SafeNet and Custodial Interoperability Approaches*. https://cpl.thalesgroup.com/access-management/security-applications/authentication-client-token-management. Accessed: 2025-09-03. 2024.

[TMM22] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. "Universal Atomic Swaps: Secure Exchange of Coins Across All Blockchains". In: *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2022, pp. 1299–1316. DOI: 10.1109/SP46214.2022.9833731.

[Thy+20] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, Fritz Schmidt, and Dominique Schröder. *PayMo: Payment Channels For Monero*. Cryptology ePrint Archive, Report 2020/1441. https://eprint.iacr.org/2020/1441. 2020.

[Tiw+22] Samarth Tiwari, Michelle Yeo, Zeta Avarikioti, Iosif Salem, Krzysztof Pietrzak, and Stefan Schmid. *Wiser: Increasing Throughput in Payment Channel Networks with Transaction Aggregation*. 2022. arXiv: 2205.11597 [cs.CR].

[Tod14] Peter Todd. *BIP-0065: OP_CHECKLOCKTIMEVERIFY - Payment Channels*. 2014. URL: https://github.com/bitcoin/bips/blob/master/bip-0065.mediawiki#user-content-Payment_Channels.

[Uni18] Uniswap. *Uniswap Exchange Protocol*. 2018. (Visited on 12/11/2020).

[Zha+18] Yuncong Zhang, Yu Long, Zhen Liu, Zhiqiang Liu, and Dawu Gu. "Z-Channel: Scalable and Efficient Scheme in Zerocash". In: *ACISP 18*. Ed. by Willy Susilo and Guomin Yang. Vol. 10946. LNCS. Springer, Heidelberg, July 2018, pp. 687–705. DOI: 10.1007/978-3-319-93638-3_39.

[ZQG21] Liyi Zhou, Kaihua Qin, and Arthur Gervais. "A2mm: Mitigating frontrunning, transaction reordering and consensus instability in decentralized exchanges". In: *arXiv preprint arXiv:2106.07371* (2021).

A Transaction Structure

In this section, we formally describe the transaction/message structure used to realize the construction outlined in Section 3. Note that all blockchains incorporate some notion of transaction IDs and nonce values to prevent replay attacks. We assume such IDs and nonce values are used in the implementation but do not formally specify them here to keep exposition simple.

A.1 Transaction F

Transaction F_b can be broadcasted by either Bob or Alice

A.1.1 Coin Input(s)

1. $*.\mathsf{Outputs}[*]$ — Any output of a any previous transaction recorded in the ledger that can be consumed either by Alice or by Bob and is not already consumed.

A.1.2 Coin Output(s)

1. **Value:** v_b
   State: $\mathtt{pk}_\mathsf{A}, \mathtt{pk}_\mathsf{B}$
   Predicate:

```
procedure PREDICATE(call_type, args_list)
    σ1, σ2 ← args_list
    m ← pk_A|pk_B
    flag1 ← Ver(pk_A, σ1, m) = 1 and Ver(pk_B, σ2, m) = 1
    if flag1 = 1 then
        return 1
    else
        return 0
    end if
end procedure
```

A.2 Transaction W

Transaction $[\mathsf{W}_{(b,i,j)}]$ can only be broadcast by the Bob.

A.2.1 Coin Input(s)

1. $\mathsf{F}_b.\mathsf{Outputs}[0]$ Funding Transaction's output 0 i.e., Funding Transaction's only output

A.2.2 Coin Output(s)

1. **Value:** v_b
   State: $\mathtt{pk}_\mathsf{A}, \mathtt{pk}_\mathsf{B}, \Delta_\mathsf{A}, h_{r^\mathsf{B}_{i-1,a_{i-1}}}, h_{r^\mathsf{B}_{i,k}} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}, h_{r^\mathsf{B}_{i+1,k}} \forall k \in \{1,\ldots,\mathsf{N}\}$
   Predicate:

```
procedure PREDICATE(call_type, args_list)
    if call_type = 1 then                                   ▷ invoked by Bob using [X_(b,i,j)]_(A,B)
        σ1, σ2, r^B_{i-1,a_{i-1}}, r^B_{i,k} ∀k ∈ {1,...,N} \ {j}, r^B_{i+1,k} ∀k ∈ {1,...,N} ← args_list
        m ← H(r^B_{i-1,a_{i-1}}) ‖ H(r^B_{i,k}) ∀k ∈ {1,...,N} \ {j} ‖ H(r^B_{i+1,k}) ∀k ∈ {1,...,N}
        flag1 ← Ver(pk_A, σ1, m) = 1 and Ver(pk_B, σ2, m) = 1
        flag2 ← H(r^B_{i-1,a_{i-1}}) = h_{r^B_{i-1,a_{i-1}}} and H(r^B_{i,k}) = h_{r^B_{i,k}} ∀k ∈ {1,...,N} \ {j}
                and H(r^B_{i+1,k}) = h_{r^B_{i+1,k}} ∀k ∈ {1,...,N}
        if flag1 = 1 and flag2 = 1 then
            return 1
        else
            return 0
        end if
    else if call_type = 2 then                              ▷ invoked by Alice after Δ_A
        σ1 ← args_list
        m ← pk_A
        if Ver(pk_A, σ1, m) = 1 and now > Δ_A then
            return 1
        else
```

return 0
end if
end if
end procedure

A.3 Transaction X

$[\mathsf{X}_{(b,i,j)}]$ can only be broadcast by the Bob.

A.3.1 Coin Input(s)

1. $[\mathsf{W}_{(b,i,j)}].\mathsf{Outputs}[0]$ Transaction W's output 0 i.e., $[\mathsf{W}_{(b,i,j)}]$'s only output

A.3.2 Coin Output(s)

1. **Value:** $\nu^{\mathsf{A}}_{b,i,j}$
   State: $\mathtt{pk}_{\mathsf{A}}$
   Predicate:
   procedure PREDICATE($args\_list$)
   $\sigma_1 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$ **then**
   return 1
   else
   return 0
   end if
   end procedure
2. **Value:** $\nu^{\mathsf{B}}_{b,i,j}$
   State: $\mathtt{pk}_{\mathsf{A}}, \mathtt{pk}_{\mathsf{B}}, \Delta_{\mathsf{B}}, h_{r^{\mathsf{B}}_{i,j}}$
   Predicate:
   procedure PREDICATE($call\_type, args\_list$)
   if $call\_type = 1$ **then** ▷ Alice complains that Bob is cheating by showing preimage of $h_{r^{\mathsf{B}}_{i,j}}$
   $\sigma_1, r^{\mathsf{B}}_{i,j} \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}} \| \mathcal{H}(r^{\mathsf{B}}_{i,j})$
   $flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$
   $flag_2 \leftarrow \mathcal{H}(r^{\mathsf{B}}_{i,j}) = h_{r^{\mathsf{B}}_{i,j}}$
   if $flag_1 = 1$ and $flag_2 = 1$ **then**
   return 1
   else
   return 0
   end if
   else if $call\_type = 2$ **then** ▷ invoked by Bob after Δ_{B}
   $\sigma_2 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{B}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{B}}, \sigma_2, m) = 1$ and $now > \Delta_{\mathsf{B}}$ **then**
   return 1
   else
   return 0
   end if
   end if
   end procedure

A.4 Transaction T

$[\mathsf{T}_{(b,i,j)}]$ can only be broadcast by the Alice.

A.4.1 Coin Input(s)

1. $\mathsf{F}_b.\mathsf{Outputs}[0]$ Funding Transaction's output 0 i.e., Funding Transaction's only output

A.4.2 Coin Output(s)

1. **Value:** v_b
   State: $\mathtt{pk}_\mathsf{A}, \mathtt{pk}_\mathsf{B}, \Delta_\mathsf{A}, h_{r^\mathsf{B}_{i-1,a_{i-1}}}, h_{r^\mathsf{B}_{i,k}} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}, h_{r^\mathsf{B}_{i+1,k}} \forall k \in \{1,\ldots,\mathsf{N}\}, h_{r^\mathsf{A}_{i+1}}$
   Predicate:

procedure PREDICATE($call\_type, args\_list$)
if $call\_type = 1$ **then** ▷ Bob settles at i using $[\mathsf{Y}\circ_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$
▷ identical to invoking $[\mathsf{X}_{(b,i,j)}]$ on $[\mathsf{W}_{(b,i,j)}]$
$\sigma_1, \sigma_2, r^\mathsf{B}_{i-1,a_{i-1}}, r^\mathsf{B}_{i,k} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}, r^\mathsf{B}_{i+1,k} \forall k \in \{1,\ldots,\mathsf{N}\} \leftarrow args\_list$
$m \leftarrow \mathcal{H}(r^\mathsf{B}_{i-1,a_{i-1}}) \| \mathcal{H}(r^\mathsf{B}_{i,k}) \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\} \| \mathcal{H}(r^\mathsf{B}_{i+1,k}) \forall k \in \{1,\ldots,\mathsf{N}\}$
$flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_\mathsf{A}, \sigma_1, m) = 1$ and $\mathtt{Ver}(\mathtt{pk}_\mathsf{B}, \sigma_2, m) = 1$
$flag_2 \leftarrow \mathcal{H}(r^\mathsf{B}_{i-1,a_{i-1}}) = h_{r^\mathsf{B}_{i-1,a_{i-1}}}$ and $\mathcal{H}(r^\mathsf{B}_{i,k}) = h_{r^\mathsf{B}_{i,k}} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}$
and $\mathcal{H}(r^\mathsf{B}_{i+1,k}) = h_{r^\mathsf{B}_{i+1,k}} \forall k \in \{1,\ldots,\mathsf{N}\}$
if $flag_1 = 1$ and $flag_2 = 1$ **then**
return 1
else
return 0
end if
else if $call\_type = 2$ **then** ▷ Bob settles at i using $[\mathsf{Y}+_{(b,i,j)}]_{(\mathsf{A},\mathsf{B})}$
▷ identical to $call\_type = 2$ above
$\sigma_1, \sigma_2, r^\mathsf{B}_{i,a_i}, r^\mathsf{B}_{i+1,k} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\} \leftarrow args\_list$
$m \leftarrow \mathcal{H}(r^\mathsf{B}_{i,a_i}) \| \mathcal{H}(r^\mathsf{B}_{i+1,k}) \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}$
$flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_\mathsf{A}, \sigma_1, m) = 1$ and $\mathtt{Ver}(\mathtt{pk}_\mathsf{B}, \sigma_2, m) = 1$
$flag_2 \leftarrow \mathcal{H}(r^\mathsf{B}_{i,a_i}) = h_{r^\mathsf{B}_{i,a_i}}$ and $\mathcal{H}(r^\mathsf{B}_{i,k}) = h_{r^\mathsf{B}_{i,k}} \forall k \in \{1,\ldots,\mathsf{N}\} \setminus \{j\}$
if $flag_1 = 1$ and $flag_2 = 1$ **then**
return 1
else
return 0
end if
else if $call\_type = 3$ **then** ▷ Bob complains that Alice is cheating by showing preimage of $h_{r^\mathsf{A}_{i+1}}$
$\sigma_2, r^\mathsf{A}_{i+1} \leftarrow args\_list$
$m \leftarrow \mathtt{pk}_\mathsf{B} \| \mathcal{H}(r^\mathsf{A}_{i+1})$
$flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_\mathsf{B}, \sigma_2, m) = 1$
$flag_2 \leftarrow \mathcal{H}(r^\mathsf{A}_{i+1}) = h_{r^\mathsf{A}_{i+1}}$
if $flag_1 = 1$ and $flag_2 = 1$ **then**
return 1
else
return 0
end if
else if $call\_type = 4$ **then** ▷ unresponsive Bob, Alice invokes after Δ_A and claims full balance
$\sigma_1 \leftarrow args\_list$
$m \leftarrow \mathtt{pk}_\mathsf{A}$
if $\mathtt{Ver}(\mathtt{pk}_\mathsf{A}, \sigma_1, m) = 1$ and $now > \Delta_\mathsf{A}$ **then**

return 1
else
return 0
end if
end if
end procedure

A.5 Transaction $\mathsf{Y}\circ$

$[\mathsf{Y}\circ_{(b,i,j)}]$ can only be broadcast by Bob. It takes input from $[\mathsf{T}_{(b,i,j)}]$ and settles the channel at state i, awarding $\nu^{\mathsf{A}}_{b,i}$ to Alice and $\nu^{\mathsf{B}}_{b,i}$ to Bob. This transaction is identical to $[\mathsf{X}_{(b,i,j)}]$ except that this transaction's input comes from $[\mathsf{T}_{(b,i,j)}]$ instead of $[\mathsf{W}_{(b,i,j)}]$.

A.5.1 Coin Input(s)

1. $[\mathsf{T}_{(b,i,j)}].\mathsf{Outputs}[0]$ Transaction T's output 0 i.e., $[\mathsf{T}_{(b,i,j)}]$'s only output.

A.5.2 Coin Output(s)

1. **Value:** $\nu^{\mathsf{A}}_{b,i,j}$
   State: $\mathtt{pk}_{\mathsf{A}}$
   Predicate:
   procedure PREDICATE($args\_list$)
   $\sigma_1 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$ **then**
   return 1
   else
   return 0
   end if
   end procedure
2. **Value:** $\nu^{\mathsf{B}}_{b,i,j}$
   State: $\mathtt{pk}_{\mathsf{A}}, \mathtt{pk}_{\mathsf{B}}, \Delta_{\mathsf{B}}, h_{r^{\mathsf{B}}_{i,a_i}}$
   Predicate:
   procedure PREDICATE($call\_type, args\_list$)
   if $call\_type = 1$ **then** ▷ Alice complains that Bob is cheating by showing preimage of $h_{r^{\mathsf{B}}_{i,a_i}}$
   $\sigma_1, r^{\mathsf{B}}_{i,a_i} \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}} \| \mathcal{H}(r^{\mathsf{B}}_{i,a_i})$
   $flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$
   $flag_2 \leftarrow \mathcal{H}(r^{\mathsf{B}}_{i,a_i}) = h_{r^{\mathsf{B}}_{i,a_i}}$
   if $flag_1 = 1$ and $flag_2 = 1$ **then**
   return 1
   else
   return 0
   end if
   else if $call\_type = 2$ **then** ▷ invoked by Bob after Δ_{B}
   $\sigma_2 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{B}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{B}}, \sigma_2, m) = 1$ and $now > \Delta_{\mathsf{B}}$ **then**
   return 1
   else

return 0
end if
end if
end procedure

A.6 Transaction Y+

$[\mathsf{Y}+_{(b,i,j)}]$ can only be broadcast by the Bob. It takes input from $[\mathsf{T}_{(b,i,j)}]$ and settles the channel at state $i+1$, awarding $\nu^{\mathsf{A}}_{b,i+1}$ to Alice and $\nu^{\mathsf{B}}_{b,i+1}$ to Bob. This transaction is identical to $[\mathsf{Y}\circ_{(b,i,j)}]$ except: (1) output balances are $(\nu^{\mathsf{A}}_{b,i+1}, \nu^{\mathsf{B}}_{b,i+1})$ instead of $(\nu^{\mathsf{A}}_{b,i}, \nu^{\mathsf{B}}_{b,i}$, and (2) revocation key is $r^{\mathsf{B}}_{i+1,j}$ instead of r^{B}_{i,a_i}.

A.6.1 Coin Input(s)

1. $[\mathsf{T}_{(b,i,j)}].\mathsf{Outputs}[0]$ Transaction T's output 0 i.e., $[\mathsf{T}_{(b,i,j)}]$'s only output.

A.6.2 Coin Output(s)

1. **Value:** $\nu^{\mathsf{A}}_{b,i+1,j}$
   State: $\mathtt{pk}_{\mathsf{A}}$
   Predicate:
   procedure PREDICATE($args\_list$)
   $\sigma_1 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$ **then**
   return 1
   else
   return 0
   end if
   end procedure
2. **Value:** $\nu^{\mathsf{B}}_{b,i+1,j}$
   State: $\mathtt{pk}_{\mathsf{A}}, \mathtt{pk}_{\mathsf{B}}, \Delta_{\mathsf{B}}, h_{r^{\mathsf{B}}_{i+1,j}}$
   Predicate:
   procedure PREDICATE($call\_type, args\_list$)
   if $call\_type = 1$ **then** ▷ Alice complains that Bob is cheating by showing preimage of $h_{r^{\mathsf{B}}_{i+1,j}}$
   $\sigma_1, r^{\mathsf{B}}_{i+1,j} \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{A}} \| \mathcal{H}(r^{\mathsf{B}}_{i+1,j})$
   $flag_1 \leftarrow \mathtt{Ver}(\mathtt{pk}_{\mathsf{A}}, \sigma_1, m) = 1$
   $flag_2 \leftarrow \mathcal{H}(r^{\mathsf{B}}_{i+1,j}) = h_{r^{\mathsf{B}}_{i+1,j}}$
   if $flag_1 = 1$ and $flag_2 = 1$ **then**
   return 1
   else
   return 0
   end if
   else if $call\_type = 2$ **then** ▷ invoked by Bob after Δ_{B}
   $\sigma_2 \leftarrow args\_list$
   $m \leftarrow \mathtt{pk}_{\mathsf{B}}$
   if $\mathtt{Ver}(\mathtt{pk}_{\mathsf{B}}, \sigma_2, m) = 1$ and $now > \Delta_{\mathsf{B}}$ **then**
   return 1
   else
   return 0
   end if

end if
end procedure

B Cost Tables

N	**F**		**W**		W_A		**X**		**T**		T_A	
	vb	**C**	**vb**	**C**	**vb**	**C**	**vb**	**C**	**vb**	**C**	**vb**	**C**
1	122	0.51	150	0.62	138	0.57	215	0.89	150	0.62	175	0.73
2	122	0.51	150	0.62	144	0.60	226	0.94	150	0.62	180	0.75
4	122	0.51	150	0.62	167	0.69	270	1.12	150	0.62	215	0.89
8	122	0.51	150	0.62	213	0.88	358	1.49	150	0.62	284	1.18
16	122	0.51	150	0.62	305	1.27	534	2.22	150	0.62	422	1.75

N	T_B		$\mathsf{Y}\circ$		$\mathsf{Y}+$		$\{\mathsf{X},\mathsf{Y}\circ,\mathsf{Y}+\}_A$		$\{\mathsf{X},\mathsf{Y}\circ,\mathsf{Y}+\}_{CB}$	
	vb	**C**	**vb**	**C**	**vb**	**C**	**vb**	**C**	**vb**	**C**
1	180	0.75	252	1.05	247	1.03	131	0.54	126	0.52
2	186	0.77	263	1.09	252	1.05	132	0.55	126	0.52
4	221	0.92	319	1.32	297	1.23	131	0.54	126	0.52
8	290	1.20	430	1.78	387	1.61	132	0.55	126	0.52
16	428	1.78	652	2.71	567	2.35	132	0.55	127	0.53

Table 5: Π_{PCM} operation costs on Bitcoin. At the time of writing, the fee rate was 4.0 sat/vB (source: https://btc.network/estimate on 2025-05-16) for inclusion withing 6 block, and the BTC/USD exchange rate was $103,747.15 (source: CoinMarketCap on 2025-05-16). The column labeled **C** denotes the cost in USD, while **vB** refers to virtual bytes. The notation $\{\mathsf{X},\mathsf{Y}\circ,\mathsf{Y}+\}$ indicates that these operations incur identical costs. F is measured with only one UTXO as input.

N	F1		F2		W		W_A		X	
	Gas	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**
1	110707	0.49	61823	0.28	173888	0.77	88788	0.40	52160	0.23
2	110707	0.49	61823	0.28	231427	1.03	98432	0.44	65255	0.29
4	110707	0.49	61823	0.28	328608	1.46	114806	0.51	87481	0.39
8	110707	0.49	61823	0.28	512989	2.28	147331	0.66	132004	0.59
16	110707	0.49	61822	0.28	873821	3.89	212156	0.94	220967	0.98

N	X_A		X_{CB}		X_{CA}		T		T_A	
	Gas	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**
1	90158	0.40	89247	0.40	36744	0.16	196582	0.88	91270	0.41
2	99132	0.44	98664	0.44	36744	0.16	252136	1.12	100465	0.45
4	116177	0.52	115487	0.51	36744	0.16	351305	1.56	117288	0.52
8	148701	0.66	148013	0.66	36744	0.16	533710	2.38	149588	0.67
16	213527	0.95	212167	0.94	36744	0.16	898518	4.00	214414	0.95

N	T_B		$\mathsf{Y}\circ$		$\mathsf{Y}\circ_A$		$\mathsf{Y}\circ_{CB}$		$\mathsf{Y}\circ_{CA}$	
	Gas	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**
1	93798	0.42	52470	0.23	92158	0.41	91721	0.41	37035	0.16
2	103442	0.46	65568	0.29	101129	0.45	100916	0.45	37035	0.16
4	119819	0.53	87783	0.39	118399	0.53	117962	0.53	37035	0.16
8	152568	0.68	132296	0.59	150698	0.67	150264	0.67	37035	0.16
16	216272	0.96	221232	0.98	215525	0.96	214866	0.96	37035	0.16

N	$\mathsf{Y}+$		$\mathsf{Y}+_A$		$\mathsf{Y}+_{CB}$		$\mathsf{Y}+_{CA}$	
	Gas	**C**	**Gas**	**C**	**Gas**	**C**	**Gas**	**C**
1	73629	0.33	91916	0.41	91962	0.41	37337	0.17
2	80302	0.36	101335	0.45	100709	0.45	37337	0.17
4	94024	0.42	117708	0.52	117757	0.52	37337	0.17
8	119170	0.53	150455	0.67	150728	0.67	37337	0.17
16	163689	0.73	215283	0.96	214885	0.96	37337	0.17

Table 6: Π_{PCM} operation costs on Ethereum. At the time of writing, the ETH/USD exchange rate was $2,588.56 (source: CoinMarketCap on 2025-05-16), and the gas price was 1.72 GWEI. The column labeled **C** denotes the cost in USD.